



Player Protection: The Black Box of Logic

By

Alberto Peñalba Martos - Grade: 10

Dorje Khampa Herrezuelo Blasco - Grade: 10



UNIVERSIDAD COMPLUTENSE
MADRID

Supervised by

José Luis Vázquez Poletti

David Pacios Izquierdo

MADRID, 2025–2026

ACKNOWLEDGMENTS

To our supervisors, for their patient guidance, judgment, and the generosity with which they have accompanied us throughout this work. Without their time, their high standards, and their trust, these pages would not be what they are today.

FROM ALBERTO

To all the people who, in one way or another, have contributed to the development and completion of this work.

I also especially value the experience and learning gained during the preparation of this project.

— Alberto Peñalba Martos —

FROM DORJE

To my mother, for her quiet effort and unconditional love, for supporting me at every step and believing in me when I needed it most.

To Lulu and Poppy, faithful companions through so many hours, whose mere presence made the world kinder.

To my uncle José Luis and my aunt Toñi, for their affection and presence, for always being there, without needing words.

And to my aunt Julia, may she rest in peace, for everything she gave me and that I still carry with me; because some people never fully leave.

— Dorje Khampa Herrezuelo Blasco —

Abstract

The video game industry lives in a paradox. On one hand, it has become the most powerful engine of the digital entertainment economy, surpassing film and music combined [1]. Millions of players connect every day to compete, cooperate, and build global communities. Yet this same industry faces a persistent threat: insecurity.

Cheating, score fraud, stolen accounts, altered virtual economies, and illegal copies are now part of the gaming landscape. The root cause is structural: for years, many projects—especially in the independent and academic sectors—have relied on the client for validation, assuming that the player’s device is trustworthy. This approach has led to vulnerabilities that compromise data integrity and player trust.

This Final Degree Project proposes an alternative approach: building games with security integrated from the start by applying distributed computing principles and leveraging *serverless* cloud architectures. The goal is to move validation authority to the server and ensure data coherence, traceability, and authenticity through reproducible and scalable mechanisms. The result is a framework that enables developers to create secure and sustainable gaming experiences without sacrificing agility.

Keywords

Cybersecurity, video games, *serverless* architecture, distributed computing, AWS, DRM, anti-cheat, validation, secure backend, cloud computing.

Contents

Chapter 1	Introduction	1
Chapter 2	State of the Art	7
Chapter 3	Technologies Used and Methodology	13
Chapter 4	Architecture and Design	21
Chapter 5	Use Cases and Observation	33
Chapter 6	Conclusions and Future Work	43
	Appendices	47
	Bibliography	56

Chapter 1. Introduction

1.1. Motivation

The video game industry constitutes one of the driving forces of the contemporary digital economy. Historically, it has surpassed the film and music industries combined in both revenue volume and global reach [1]. What emerged as a domestic *hobby* in the early 1970s has become a cultural, ecosocial, and technological phenomenon, supported by international communities, high-performance competitions (*eSports*), and complex virtual economies.

Fraudulent score alterations, the injection of *cheats*, manipulation of client state, and the distribution of unauthorised copies represent everyday threat vectors. Legitimate users trust in the fairness of the interactive experience and in their progression being assessed rigorously. Nevertheless, much of the infrastructure that supports this trust is trivially exploitable because of underlying architectural vulnerabilities.

This situation not only damages the way games are played, but also harms the reputation of developers and player communities.

Historically, various countermeasures have been deployed to mitigate these vulnerabilities. One of the first lines of defence consisted of integrating *Digital Rights Management (DRM)* mechanisms. These mechanisms were conceived to restrict unauthorised copying and execution of *software*. The evolution from simple activation-key checks to complex dynamic obfuscators such as *Denuvo Anti-Tamper* evidences an ongoing arms race between intellectual-property protection and reverse engineering.

In parallel, the development of intrusion-detection systems specific to video games (*anti-cheat*) has proliferated. These solutions have undergone notable sophistication. Whereas in their initial iterations they were limited to static verification in user space, they now predominantly operate at the *kernel* level of the operating system (for example, *BattlEye* or *Easy Anti-Cheat*), theoretically increasing friction against memory manipulation.

Even so, these mechanisms share a structural limitation: they rely heavily on the integrity of the client. No matter how sophisticated they are, as long as the control point resides on the player's device, an attack surface will always remain open. The motivation for this work arises precisely from that weakness: rethinking where system authority should reside instead of relying on legitimate behaviour by the player.

This structural weakness manifests itself recurrently in practice. A documented recent case is that of February 2026, when MKDEV released a *crack* for *Resident Evil Requiem*, protected by *Denuvo Anti-Tamper*. In order to bypass the protection, the *crack* required disabling security features integrated by default in *Windows* environments, such as *Secure Boot*, *VBS*, *HVCI*, *Hyper-Visor*, and the memory integrity feature of *Windows Defender*, as well as enabling the loading of *drivers* in *Test-Mode*. This configuration leaves the machine exposed to *kernel-level rootkits* that are not detectable by either the user or the operating system; in this specific case, the distributed *crack* also included a *keylogger*. Beyond the

anecdote itself, the episode illustrates a more general pattern: when the control point resides on the client, the costs that the user assumes in order to circumvent protection may end up compromising the security of the operating system itself.

It is therefore proposed that intrinsically more secure game topologies can be designed by adopting principles of distributed computing and *serverless*¹ architectures, shifting logical validation authority and critical state decisions from the client to the *backend*. This paradigm shift not only safeguards session integrity, but also establishes an auditable, deterministic, and sustainable framework.

Problem and gap

The underlying problem does not lie in a lack of cryptographic or technological primitives, but in the prevailing architectural paradigm. In many developments — particularly in *indie* or academic environments — there is an overdependence on client integrity: scores, movement vectors, or inventories originating on the user’s machine are validated *de facto* without subsequent authoritative reconciliation on the server. Although this *client-authoritative* model accelerates the initial development cycle, it introduces a massive attack vector that compromises the long-term viability of the ecosystem.

The result of this trend is a fragmented and uneven ecosystem. Games end up being only as secure as the weakest user device. Data tampering, *exploits*, and cheating cease to be isolated incidents and become symptoms of a poorly conceived architecture. Furthermore, dependence on local solutions prevents systems from being scaled or maintained coherently, increasing technical debt and operational risk.

A twofold gap can therefore be identified:

- **Technical gap:** generated by the lack of reference architectures (or *blueprints*) that democratise the deployment of *server-authoritative* solutions in teams with limited resources.
- **Methodological gap:** caused by treating security as a final stage of development rather than as a design principle (*security by design*).

This work seeks to contribute to closing that gap by defining an architectural approach that prioritises server-side verification, observability, and data consistency, without sacrificing agility or the quality of the experience, always within a *serverless* environment.

Scope and context of the work

This Final Degree Project does not aim to solve every security problem in the industry, but rather to address a specific and representative area: the protection of the critical elements of a video game through a secure and scalable *backend*².

The scope is delimited around the following axes:

¹A cloud execution model in which infrastructure provisioning and scaling are managed dynamically by the provider, abstracting the physical layer.

²The part of an application or website that remains invisible to the end user.

- It focuses on the design and evaluation of an architecture that manages licences, progression, scores, and authentication from the server.
- A *kernel-level anti-cheat* system is not developed, although its relevance is acknowledged as part of the historical context.
- New *DRM* algorithms are not designed, but they are analysed as precedents in client-integrity control.
- Real-time multiplayer networks and latency-related aspects are not addressed, since the focus lies on logical security and consistency of the game state.

From a contextual standpoint, this work falls within two complementary frameworks:

- **Academic and educational:** as it is a project oriented towards applied learning in cybersecurity, systems architecture, and video game development.
- **Practical and replicable:** as it offers a viable model for independent studios or small-scale projects, where security is often relegated because of time or resource constraints.

The value of this research is considered to lie precisely in this dual perspective: combining academic rigour with a pragmatic and reusable contribution.

1.2. Objectives

General objective

To design a *backend* architecture based on principles of distributed computing and *serverless* models, capable of shifting validation authority from the client to a controlled environment, thereby guaranteeing the integrity of critical video game data.

Specific objectives

- Protect the fundamental elements of the game (licences, scores, progression) against external tampering.
- Ensure player authentication and identity through centralised and verifiable mechanisms.
- Implement validation flows that guarantee the consistency of the data sent by the client.
- Establish logs and traceability mechanisms that make it possible to detect irregularities and audit actions.
- Develop a conceptual model that may serve as a replicable reference for future projects.

These objectives aim to build a framework that prioritises trust, verifiability, and system sustainability over technical complexity.

1.3. Working hypotheses

Although hypotheses are traditionally associated with empirical or experimental approaches, in this case they are used as formulations of intent, equivalent to verifiable objectives. It is proposed that:

- An architecture that centralises validation on the server reduces the tampering of scores and progression.
- The incorporation of structured telemetry makes it possible to detect anomalous behaviour at an early stage.
- Integrating security from the design stage reduces technical debt and improves project maintainability.

These statements guide the development of the work and orient the validation of its results.

1.4. Methodological approach

The work will be carried out following a structured and progressive methodology:

1. **Domain and risk analysis:** identification of the most sensitive assets, the most common threats, and recurrent insecure practices.
2. **Conceptual architecture design:** definition of the authenticate–authorise–validate–log flows and of the applicable distributed-computing principles.
3. **Prototype implementation:** development of a controlled environment in which the proposed measures are evaluated.
4. **Instrumentation and observability:** incorporation of logging and telemetry mechanisms to assess coherence and behaviour.
5. **Experimental evaluation:** tests under simulated adverse scenarios that allow the effectiveness of the approach to be measured.
6. **Synthesis of results:** extraction of conclusions and formulation of a *blueprint* applicable to other contexts.

1.5. Expected results

As a result of carrying out this work, the following outcomes are expected:

- A reproducible architecture that demonstrates the feasibility of shifting validation authority to the server.
- A set of best practices applicable to educational or independent projects.
- Experimental evidence that centralised verification improves data consistency and reduces tampering risks.

- A methodological contribution that promotes the integration of security from the earliest design phases.

In this way, the aim is to contribute to a more responsible and sustainable view of video game development, one in which security ceases to be an obstacle and becomes a structural principle.

Chapter 2. State of the Art

2.1. Introduction

Modern video games depend on increasingly complex and distributed infrastructures. Behind every multiplayer match, progression system, or live event there is a technical fabric that combines game engines, cloud services, content delivery networks, and global databases. This transformation has enabled real-time experiences for millions of users, but it has also multiplied the attack surface.

The shift from local entertainment to connected environments has changed the rules of development. Today, the client is no longer an isolated executable, but an entry point into persistent ecosystems that manage identities, virtual economies, and sensitive data. This hyperconnectivity has turned security from an optional add-on into a structural design principle.

The objective of this chapter is to examine how the video game industry has addressed cybersecurity from different historical and contemporary approaches, identifying the limitations of traditional models based on client trust and analysing the opportunities offered by distributed and *serverless* architectures. It also presents the theoretical and regulatory frameworks that guide security best practices in cloud environments, together with emerging trends that seek to integrate security as a verifiable and automated component.

2.2. Risks and vulnerabilities in connected video games

The contemporary interconnected game ecosystem inherits substantial vulnerabilities from web and mobile applications, but amplifies them through real-time interactivity and the global exposure of its services. The classic client-server architecture, widely adopted since the 1990s, starts from an unsafe assumption: the client will behave honestly. In practice, this trust translates into multiple points of attack.

Client manipulation

Client manipulation includes techniques such as memory modification, dynamic library injection (*DLL¹ injection*), executable patching, or the use of debugging tools to alter game logic. These attacks make it possible to modify internal variables, accelerate movement, alter physics, or even disable local checks.

In competitive titles such as *Counter-Strike: Global Offensive* or *Fortnite*, cheats based on *memory editing* and *aimbots* have become a structural problem. Although *anti-cheat* systems attempt to detect these alterations, constant software updates and attacker creativity create an endless race.

Recent studies such as that of Prazak et al. [2] show that telemetry can be an effective tool for detecting anomalies in player behaviour through statistical analysis and machine

¹Dynamic Link Library, a term used for files containing executable code that are loaded on demand by a program through the operating system.

learning. However, this approach is usually reactive and requires an architecture capable of collecting and processing large volumes of data in real time.

API and communication abuse

Another critical source of vulnerabilities is found in *backend APIs*²³. If adequate controls are not implemented, these interfaces can be exploited to alter scores, progress, or inventories. Documented cases in mobile game communities and independent platforms show how tools such as *Fiddler* or *Charles Proxy* allow *HTTP*⁴ traffic to be intercepted and requests modified before they reach the server.

According to the *OWASP*⁵ *Foundation* [3], the most common vulnerabilities in *APIs* include weak authentication, excessive data exposure, lack of request limits, and object-level authorization errors. These weaknesses are especially serious in game environments, where *endpoints*⁶ often handle economic or persistent progression data.

Weak identity and session management

In modern video games, digital identities are valuable assets. Compromised accounts can provide access to purchases, achievements, and progress. Password reuse, lack of multi-factor authentication, and *tokens*⁷ without expiration are recurring problems.

Services such as *Cognito* [4] and schemes based on *OAuth 2.0* and *JWT*⁸ offer robust mechanisms for federated authentication and session control.

Operational risks

Beyond the technical layer, modern video games face operational risks derived from their dynamic nature. Live updates, rapid patches, and temporary events constantly modify the attack surface. A misconfigured access rule in a deployment can expose sensitive data or internal *APIs*. Security, in this context, cannot be understood as a state, but as a continuous process of review, auditing, and observability.

2.3. Traditional protection strategies

Over the last three decades, the video game industry has developed various strategies to protect its software and integrity. The most prominent are *Digital Rights Management (DRM)* systems and *anti-cheat* systems, which, although different in purpose, share a common dependence on the client environment.

²Application Programming Interface, a set of rules, protocols, and tools that allows different software applications to communicate with each other and exchange data in a standardised and defined way.

³The hidden part of a web application or software system that runs on the server.

⁴Hypertext Transfer Protocol, the fundamental protocol of the World Wide Web that defines the rules for communication between web browsers and servers.

⁵Open Web Application Security Project, an open-source project dedicated to identifying and combating the causes that make software insecure.

⁶A point where a system exposes functionality that another system can access remotely.

⁷A digital mechanism that verifies the identity of a user or device without repeatedly using traditional credentials.

⁸JSON Web Token, a standard format (RFC 7519) for transmitting authenticated information between two parties through a digitally signed *JSON* object.

Digital Rights Management (DRM)

DRM mechanisms emerged as a direct response to piracy and unauthorised distribution. From the classic activation serials of the 1990s to modern online validation schemes, DRM has attempted to ensure that each copy of a game is legitimate.

Early examples include the activation system of *Microsoft Games for Windows Live* or *CD-Key* validations in *Blizzard*. With the expansion of digital platforms such as *Steam*, *Origin*, and *Epic Games Store*, DRM became part of the ecosystem, integrating online verification as an execution requirement.

The most emblematic case is *Denuvo Anti-Tamper* [5], designed to prevent reverse engineering and protect the executable. Although effective at delaying piracy, it has generated debate because of its impact on performance and its implications for the user experience [6]. DRM illustrates the classic dilemma between control and accessibility: the more restrictive the system, the greater the friction with the end user.

Anti-cheat systems

Anti-cheat systems aim to preserve fairness in competitive environments. Tools such as *BattlEye* and *Easy Anti-Cheat* [7] operate at a low level of the operating system, monitoring processes, system calls, and code signatures to detect cheats.

There are two main approaches: (1) heuristic detection, based on predefined rules and blacklists; and (2) behavioural detection, supported by telemetry analysis and machine learning.

Kernel-level systems, such as those used by *Valorant* or *Fortnite*, offer deep control but raise serious challenges related to privacy, compatibility, and maintenance. Moreover, they continue to share the fundamental problem: they depend on a client that can be manipulated.

2.4. Evolution of network topologies in video games

The evolution of network topologies in video games reflects a progressive shift of authority from clients towards more controlled infrastructures. Each step reduces certain attack vectors, although it introduces new operational and observability responsibilities.

Figure 2.1 summarises this evolution and compares the security implications of each model.

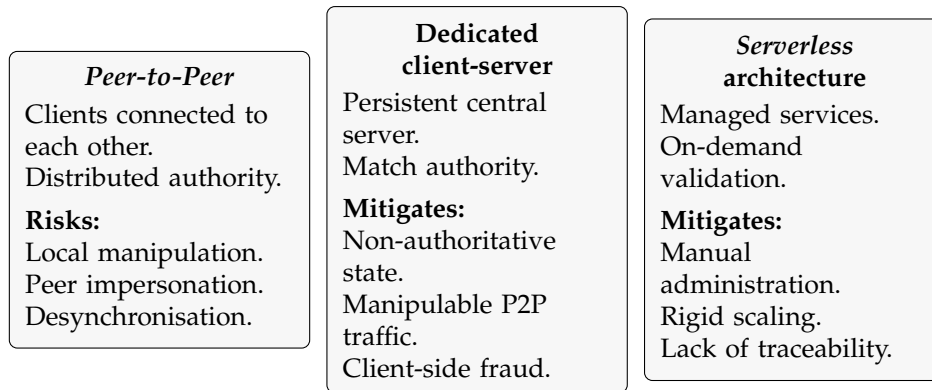


Figure 2.1: Comparison between *Peer-to-Peer* topology, dedicated client-server topology, and *serverless* architecture.

Cloud computing has radically changed how video game *backends* are conceived. Compared with the static infrastructures of the past, it is now natural to rely on platforms that scale resources on demand. Three models are usually distinguished:

- *IaaS (Infrastructure as a Service)*: the provider offers virtual machines and storage, while the user manages the operating system and applications.
- *PaaS (Platform as a Service)*: the execution environment is already managed, so the application simply needs to be deployed.
- *FaaS (Function as a Service)*: also known as the *serverless* model, it executes code in response to events and charges only for performed invocations.

In video games, the *serverless* approach fits especially well with the objective of moving authority from the client to the server: validating progress, licences, or scores in an environment beyond the player's reach. *Lambda* [8] handles business logic, *API Gateway* [9] acts as the entry point with authentication and traffic control, *Cognito* [4] manages identity and session, and *DynamoDB* [10] provides the underlying storage.

By avoiding the exposure of self-managed servers to the Internet, the attack surface is reduced considerably. System security configuration and review have been supported by references such as the *CIS AWS⁹ Foundations Benchmark* [11] and the *Zero Trust* model [12]. AWS is the cloud platform on which the project is built, and it provides the compute, storage, database, and networking services used in the architecture.

The *serverless* model is not without drawbacks: event traceability becomes more complex, strong provider dependence appears, and observability becomes a first-order requirement. This explains the importance of auditing tools: *CloudWatch*¹⁰, responsible for collecting metrics, *logs*, and events from both AWS resources and deployed applications, and *AWS X-Ray*, which makes it possible to trace each request throughout the architecture.

⁹Amazon Web Services, a cloud computing platform that offers compute, storage, database, and networking services on demand.

¹⁰An AWS monitoring and observability service that collects and tracks metrics, *logs*, and events from AWS resources, cloud applications, and local environments.

2.5. Security frameworks and design principles

The secure design of distributed systems is supported by consolidated theoretical principles that guide the practical implementation of protection measures.

Zero Trust

The *Zero Trust* paradigm, defined by NIST [12], establishes that no user, service, or device should be considered trustworthy by default. Each request must be authenticated, authorised, and verified according to context. Applying this principle to video games means validating every action sent from the client, ensuring that the server always maintains authority over the game state.

Security by design and *DevSecOps*

The *security by design* approach promotes the early integration of cybersecurity into the development cycle. In video games, this means including validations, logs, and identity controls from the prototyping phase.

DevSecOps methodologies extend this concept by incorporating test automation, static code analysis, and controlled deployments with continuous verification. Platforms such as *AWS CodePipeline* or *GitHub Actions* make it possible to integrate security scans and audits before each deployment, reducing the likelihood of human error.

Regulations and standards

Organisations such as *OWASP* and the *Center for Internet Security (CIS)* [11, 13] offer widely accepted reference frameworks. The adoption of configuration and audit controls makes it possible to align architectures with industry standards. In particular, the *CIS Controls v8* establish fundamental practices such as identity management, centralised logging, and privilege minimisation.

2.6. Identified gap and emerging trends

Despite the existence of tools, guidelines, and consolidated principles, a significant gap persists between theory and practice. In academic or independent development environments, security is often viewed as an obstacle or an added cost, resulting in projects that prioritise gameplay over *backend* robustness.

This gap manifests itself on two levels: (1) a technical gap, related to the absence of reproducible frameworks that facilitate the adoption of cloud security; and (2) a methodological gap, linked to the lack of a security culture integrated into development.

Video game security is now moving towards a hybrid model. Large-scale titles such as *Valorant* and *Overwatch* already combine telemetry analysis [2] with machine learning to detect atypical actions on the client and server. All indications suggest that security will cease to be reactive and will come to depend on what is observable and verifiable through *serverless* architectures.

This work therefore positions itself in the two least replaceable quadrants: **server-side validation** and **audited observability**. It does not aim to compete with *DRM* or client-side *anti-cheat*, but rather to cover what neither solves at the root: ensuring that

the authorised state of the game (licences, sessions, scores, progression) is intact and auditable. According to the end-to-end principle [14], that authority can only be exercised on the server. The counterpart, assumed from the outset, is that purely perceptual cheats remain outside the scope of this work and are treated as a separate problem.

In this sense, the work is not presented as a replacement for existing models, but as a component that fills a specific gap in the map: offering a reproducible *blueprint*, based on *serverless* over *AWS*, that centralises authentication, validation, and traceability, and that remains accessible for independent projects.

Chapter 3. Technologies Used and Methodology

3.1. Architecture overview

The prototype is structured as a *serverless backend* for a video game developed in *Unity*, accompanied by a self-service *web* portal for registration and licence management. The adoption of the *serverless* model does not respond to a generic technological preference, but to two specific constraints of the project: isolating sensitive logic in a controlled environment, controlling the *demo* mode, and verifying executable integrity.

The *Unity* client (C#) and the *HTML/JS* licence-management page share the same *Cognito user pool* and communicate with the *backend* through an *HTTP API* exposed by *API Gateway*. This gateway validates at the perimeter the *JWT tokens* issued by *Cognito* before delegating the request to a *Lambda* function. Each *Lambda* implements a specific domain operation and persists its state in independent *DynamoDB* tables. The game executable is served from *S3*, and *CloudWatch* centralises the logs and metrics on which system observability relies.

The detailed justification for each choice is addressed in the following section.

3.2. Technology justification

3.2.1. Entry layer: *API Gateway*

API Gateway acts as the system entry point: it receives requests from the *Unity* client and from the web portal, and redirects them to the corresponding *Lambda*.

This service was chosen for three reasons. Its integration with *Cognito* makes it possible to validate *JWT tokens* at the perimeter itself, checking signature, expiration, and issuer before the request reaches the *backend*; if something does not match, the request is discarded without invoking any *Lambda*. In addition, each method supports a *JSON*¹ schema as an input contract, which filters malformed requests from the start and avoids having to repeat that validation in each function. Finally, configurable limits by *endpoint*, identity, or *IP*² protect the most sensitive operations, such as score submission or executable verification, without requiring external components.

3.2.2. Logic layer: *Lambda*

The *backend* logic is divided into several *Lambda* functions, each with its own responsibility: registering sessions, validating statistics, checking the executable *hash*³, controlling demo mode, issuing licences, and generating pre-signed *URLs*⁴. Each *API*

¹JavaScript Object Notation, a lightweight data-interchange format based on key-value pairs.

²Internet Protocol, the numerical address that identifies each device connected to a network.

³A fixed-length string obtained by applying a mathematical function to input data; it is used to verify data integrity, since any modification produces a different value.

⁴Uniform Resource Locator, a string that uniquely identifies a resource accessible through the web.

Gateway endpoint calls a single function, so permissions can be adjusted case by case: a *Lambda* only accesses the *DynamoDB* tables and *S3 buckets* it needs.

Because *Lambda* functions run without state, invocations are independent from one another. This prevents an attacker from maintaining an active process between requests and ensures that, if one invocation fails, the others are not affected.

3.2.3. Data layer: *DynamoDB*

The prototype state is stored in several *DynamoDB* tables, one per domain: player profile and progress, sessions, *demo* mode control, executable-integrity verification, and event logging. The queries required by the system are simple — lookups by user identifier, session identifier, or *hash* — so a key-value model covers the prototype use cases without requiring a relational engine such as *Aurora*, which would be more expensive to maintain and disproportionate at this scale.

Another useful advantage is the *TTL*⁵ attribute, which makes it possible to automatically delete ephemeral data, such as single-use *nonces* or expired *tokens*, without adding periodic cleanup tasks.

3.2.4. Storage and distribution layer: *S3*

S3 is used as storage for the game executable on both *macOS* and *Windows*. The *bucket* does not expose objects directly: each download goes through a pre-signed *URL* generated by a *Lambda* and expiring after a short time, so the content is never publicly accessible and credentials do not need to be distributed from the client.

Access control fits this workflow. Only the functions that generate the links have read permission over the objects; the *Unity* client and the rest of the system access them only through signed *URLs*.

3.2.5. Identity layer: *Cognito*

Identity management is delegated to *Cognito* through a single *user pool* shared by the *Unity* client and the web portal. The *backend* never stores passwords: authentication passes entirely through *Cognito*, which uses the *SRP*⁶ protocol so credentials do not travel in clear text.

Once authenticated, the user receives three signed *JWT tokens* — *access token*, *ID token*, and *refresh token* — that *API Gateway* validates on every request. The *access token* expires quickly, so the client is responsible for requesting renewal in the background using the *refresh token*, without asking the player for credentials again. If suspicious activity is detected, revoking the *refresh token* is enough to cut off access for a session.

3.2.6. Observability layer: *CloudWatch*

CloudWatch centralises the *logs* of *Lambda* functions and the operational metrics that the prototype uses to know its state at all times. The functions emit structured records

⁵Time To Live, a mechanism that assigns a time limit to data, after which it is automatically deleted.

⁶Secure Remote Password, a cryptographic protocol for password authentication without transmitting passwords in clear text.

that make it possible to reconstruct the path of each request from its entry through *API Gateway* to its persistence in *DynamoDB*.

From these records, the metrics displayed in the dashboards are derived: *p50* and *p95* latencies, 4XX and 5XX error counters, requests per second by *endpoint*, and failed validations by *JSON* schema or *token*.

3.3. Data design and contracts

3.3.1. Data model

Instead of centralising all state in a single table, the design separates information into several independent *DynamoDB* tables: player profile and progress, active sessions, demo-mode control, executable-integrity verification, and event logging. This separation clarifies responsibilities and allows each *Lambda* to access only the subset of data it needs.

Each table is modelled around its real queries. The sessions table is queried by session identifier, the demo-mode table records timestamps and accumulated time per user, and the *hashes* table stores the values used to validate the version executed by the client.

3.3.2. Event schema

Lambda functions log each action with a common structure. Some of the main fields are:

- **timestamp**: when the event occurred, in milliseconds.
- **userID**: the user who performed it, according to the *Cognito* identity.
- **IP**: the address from which the request was made.
- **deviceID**: the device from which the user connected.

Each event is also classified by severity level:

- **REPORT**: development diagnostics (RequestID, execution time, memory).
- **INFO**: normal system operation.
- **WARNING**: situations requiring attention, such as repeated authentication failures.
- **ERROR**: failures that affect a specific functionality without stopping the system.

3.3.3. API contracts

The *backend* exposes a reduced set of *endpoints*, each associated with a specific game operation:

- **GET /get-game**: base information to start the application.
- **POST /register-session**: registration of a new game session.

- POST /verify-stats: validation of the statistics sent by the client.
- POST /check-version: check of the active game version.
- GET /demoCheck: temporary status of demo-mode access.
- ANY /VerifyGameHash: validation of the integrity of the executed version.

Each *endpoint* has an associated *JSON Schema*⁷ that defines required fields, expected types, and request restrictions. Requests that do not fit their contract are rejected in *API Gateway*, before invoking the corresponding *Lambda*.

Responses follow a unified structure: the *HTTP* code reflects the general connection status and the message body adds a business code that the *Unity* client uses to distinguish specific situations. This allows the game to react to each case without having to interpret *HTTP* codes ambiguously.

3.4. Critical system flows

3.4.1. User registration and identity management

User management begins on a registration website that shares the same *Cognito user pool* as the rest of the system. The process is intuitive for the player: they create an account by entering their basic information, and the system automatically validates that the credentials are correct and that no duplicate records exist.

After confirming the identity through a verification code sent by email, that account becomes the user's unique key. With this same identity, the player can sign in smoothly both on the web portal and in the game, allowing the *backend* to recognise and manage each profile individually and securely at all times.

From that moment on, the application works with an already authenticated identity, allowing the rest of the *backend* functions to operate on a unique identifier verified by *Cognito*. In this way, licence management does not depend on an additional manual registration process, but on the existence of a valid and activated account in the centralised authentication system.

3.4.2. Score submission

Score submission includes several measures to prevent abuse and guarantee system protection.

With regard to data coherence, the server validates that submitted scores make sense. They are compared with the user's progress, or checked to ensure that there are no large jumps, in order to identify atypical or manipulated values.

All attempts that violate the previous points are recorded in audit *logs*. These violations can trigger automatic alarms so that a manual review can be carried out, enabling an early response to fraudulent cases.

⁷A vocabulary that enables the consistency, validity, and interoperability of *JSON* data at scale.

3.5. Security and compliance

3.5.1. Zero Trust

The *backend* assumes that no request is trustworthy by default, not even those originating from the system itself: each access is explicitly authenticated and authorised. Authentication is covered by the *Cognito* and *API Gateway* flow described above; what *Zero Trust* adds is the set of restrictions on what each user can do once authenticated.

This is implemented through *IAM policies*⁸, which specify who can access which resource and with which actions. In practice, a player can only query their own scores, not those of others, and each *Lambda* accesses only the tables and *buckets* it needs for its task [15].

3.5.2. Surface hardening

To prevent abusive traffic from reaching the game logic, the system combines several filtering layers at the perimeter:

- *WAF*⁹: a web application firewall inspects headers, bodies, and routes, and blocks known attack patterns before the request reaches *API Gateway*.
- *Throttling*¹⁰: request limits by *IP*, user, and *token*, adjusted globally and per critical *endpoint* (score submission, authentication).
- Origin restrictions: only requests from expected origins and user agents are accepted; the rest are discarded as additional friction for illegitimate clients.
- Schema validation: each method applies a strict *JSON* contract and rejects at the perimeter any request with missing fields, incorrect types, or unexpected structures.

3.5.3. Secret management

The prototype does not incorporate a dedicated secret-management service such as *Secrets Manager* or *Parameter Store*. Instead, the strategy relies on minimising exposure: *Cognito* handles authentication, *Lambda* functions operate with temporary *tokens* and minimal *IAM* permissions, and private *S3* objects are served through pre-signed *URLs* instead of exposing *bucket* credentials.

Validation and signing logic remains entirely in the *backend*, so no key or sensitive value travels to the *Unity* client or remains visible in the distributed code.

3.5.4. Traceability and auditing

System traceability is oriented towards recording and correlating relevant *backend* actions, so that the behaviour of a request can be reconstructed from entry to persistence.

⁸Identity and Access Management policies, *JSON* documents that define which actions an identity can perform over *AWS* resources.

⁹Web Application Firewall, a security tool that protects websites and web applications by filtering and monitoring *HTTP* traffic between the application and the Internet.

¹⁰A traffic-management technique that limits the number of requests a user or application can make in a given period.

Instead of proposing a complete forensic-auditing system, the architecture prioritises operational observability: requests arrive through *API Gateway*, are processed by *Lambda* functions, and leave traces in *CloudWatch* through structured logs that facilitate debugging, error tracking, and detection of anomalous behaviour.

In addition to execution logs, part of the relevant domain state is stored in *DynamoDB* tables, making it possible to later verify decisions such as licence validations, version checks, *demo* control, or rejection of inconsistent statistics. This combination of application logs and state persistence provides enough evidence to review specific incidents and understand which decision the system made at each moment, without maintaining a separate dedicated auditing subsystem.

3.6. Methodology

The methodology combines functional iterations with security controls integrated from the design stage. Figure 3.1 summarises how design, code, testing, deployment, telemetry, and auditing are connected.

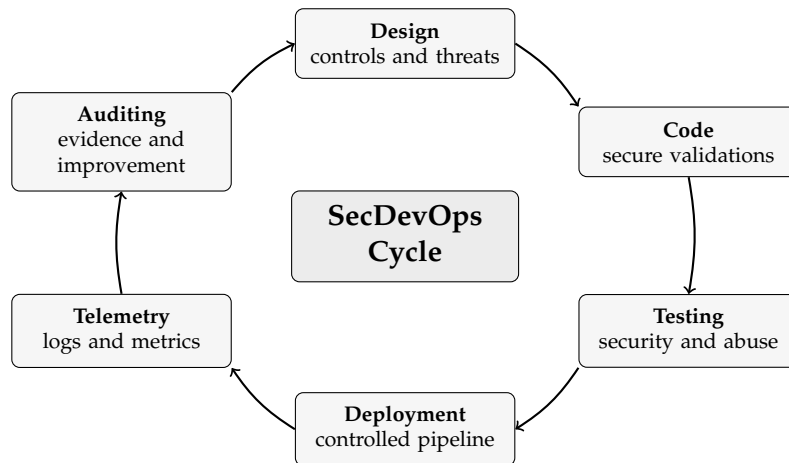


Figure 3.1: Integration of security-validation processes into the agile development cycle.

3.6.1. Incremental approach

The *backend* was developed incrementally: each iteration adds a layer of functionality and security on top of the previous one. Critical flows were prioritised — authentication, score validation, executable-integrity verification, *demo* mode control, and telemetry — while the rest were incorporated in successive cycles. This reduces risk and validates design decisions early, leaving room to correct them when needed.

Each increment results in a functional system that is deployed to a test environment and subjected to validation and observability exercises. If unexpected threats or constraints appear at that point, the architecture is adjusted before moving to the next step. The underlying idea is *security by design*: security is not added at the end of the process, but accompanies development from the beginning [16].

3.7. Privacy and ethical considerations

Privacy was considered from the beginning of the design. Since the *backend* handles identities and telemetry, it is important to define what is stored, for how long, and for what purpose. The prototype relies on three ideas:

- **What is collected.** Only what is essential: user *ID*, score, *IP* for auditing, and device fingerprint. Real names and payment information are not stored.
- **How long it is retained.** Ephemeral data, such as session *tokens* and *nonces*, is automatically deleted through *DynamoDB TTL*, so records do not remain beyond the necessary time.
- **What it is used for.** Telemetry is used to detect cheating and keep the game stable, and nothing else. It is not used to profile the player or to adjust difficulty for economic purposes.

3.8. Chapter conclusions

This chapter has argued and substantiated that a *serverless* architecture on *AWS* is the most appropriate alternative to meet both the security requirements and the scaling needs of the system. The use of *API Gateway*, *Lambda*, *DynamoDB*, *Cognito*, *S3*, and *CloudWatch* makes it possible to centralise logic in the *backend*, isolating code and sensitive elements. In this way, *Zero Trust* principles are applied while maintaining a continuous record of player and platform actions. The chapter has also emphasised that security is not a later addition, but a guiding principle that permeates every layer.

The adoption of Infrastructure as Code through *AWS Serverless Application Model* templates also ensures that the architecture is reproducible and auditable, providing a solid starting point from which to scale the *backend* in a controlled manner and without error-prone manual configurations.

Chapter 4. Architecture and Design

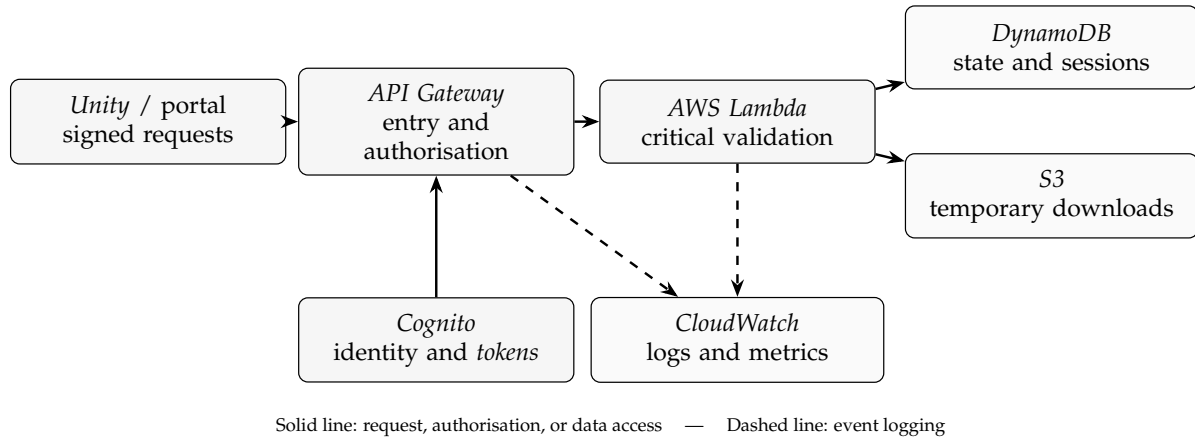


Figure 4.1: *Cloud* component topology for transactional validation.

4.1. Architecture and layer overview

As anticipated in Figure 4.1, this chapter describes the *backend* architecture proposed in the previous chapters, which transfers all validation authority from *Unity* to a *serverless* environment in *AWS*. The trust boundary is placed strictly at *API Gateway*; the client simply sends signed data, but does not perform any operation, and much less any critical operation. The architecture is organised into well-defined layers, already introduced in Chapter 3: entry layer (*API Gateway*), identity layer (*Cognito*), logic layer (*Lambda*), data layer (*DynamoDB*), distribution layer (*S3*), and observability and monitoring layer (*CloudWatch*). These layers are aligned with the objectives established in Chapter 1: data coherence, traceability through *logging*, *serverless* scalability, and agility without compromising development speed.

4.1.1. Layer decomposition

The architecture is structured into the following functional layers:

This layer decomposition preserves *Zero Trust* principles at the entry point, reduces the attack surface by encapsulating access to external services, denies any operation performed directly by the client, and facilitates the incorporation of new modules without rewriting the *backend*.

Layer	AWS Service	Main Responsibility
Entry	<i>API Gateway</i>	Secure exposure of <i>endpoints</i> and initial authorisation
Identity	<i>Cognito</i>	Federated authentication and session/ <i>token</i> management
Logic	<i>Lambda</i>	Validation of <i>hashes</i> , statistics, and critical business logic
Data	<i>DynamoDB</i>	Coherent storage of progress, sessions, and telemetry
Distribution	<i>S3</i>	Secure asset downloads through temporary pre-signed links
Observability	<i>CloudWatch</i>	Granular <i>logging</i> and real-time anomaly detection

Table 4.1: Layered structure of the proposed *serverless* architecture.

4.1.2. HTTP response convention

The *Lambda* functions return *HTTP* 200 when the business result is successful. If there is a data-processing or Internet-connection failure, an error code 400/500 is returned. On the other hand, requests that have been processed correctly but whose business result is unfavourable contain that information in the message.

- 200 OK + message: OK -; Business success
- HTTP + message: BANNED/UPDATE_REQUIRED/EXPIRED... -; Processed and denied
- 400/500 ERROR -; Technical server failure

These are some examples that *Lambda* may return depending on the case and on the function called. They are only examples used to illustrate more precisely the flows of the different *Lambda* functions detailed in this chapter.

This convention allows *Unity* to use a single entry point to process all responses.

Figure 4.2 shows the full communication flow between the *Unity* client and the *serverless backend* deployed on AWS. On the left, *Unity* sends *JSON* requests through *API Gateway*, which acts as the entry point and first validation layer. From there, each request is routed to its corresponding *Lambda* function: *CheckDemoStatus* (which queries the *DemoAccess* table), *RegisterSession* (which persists data in *PlayerStats*), *VerifyPlayerStats* (which validates metrics with *PlayerStats* if it finds a valid *nonce* in *SessionNonces*), and *VerifyGameHash* (which compares the executable *hash* with the *HistoricExeHashes* table). Responses return to the client through *API Gateway*, represented on the right. Each *Lambda* operates independently and without state, accessing only the *DynamoDB* tables assigned to it under the principle of least privilege.

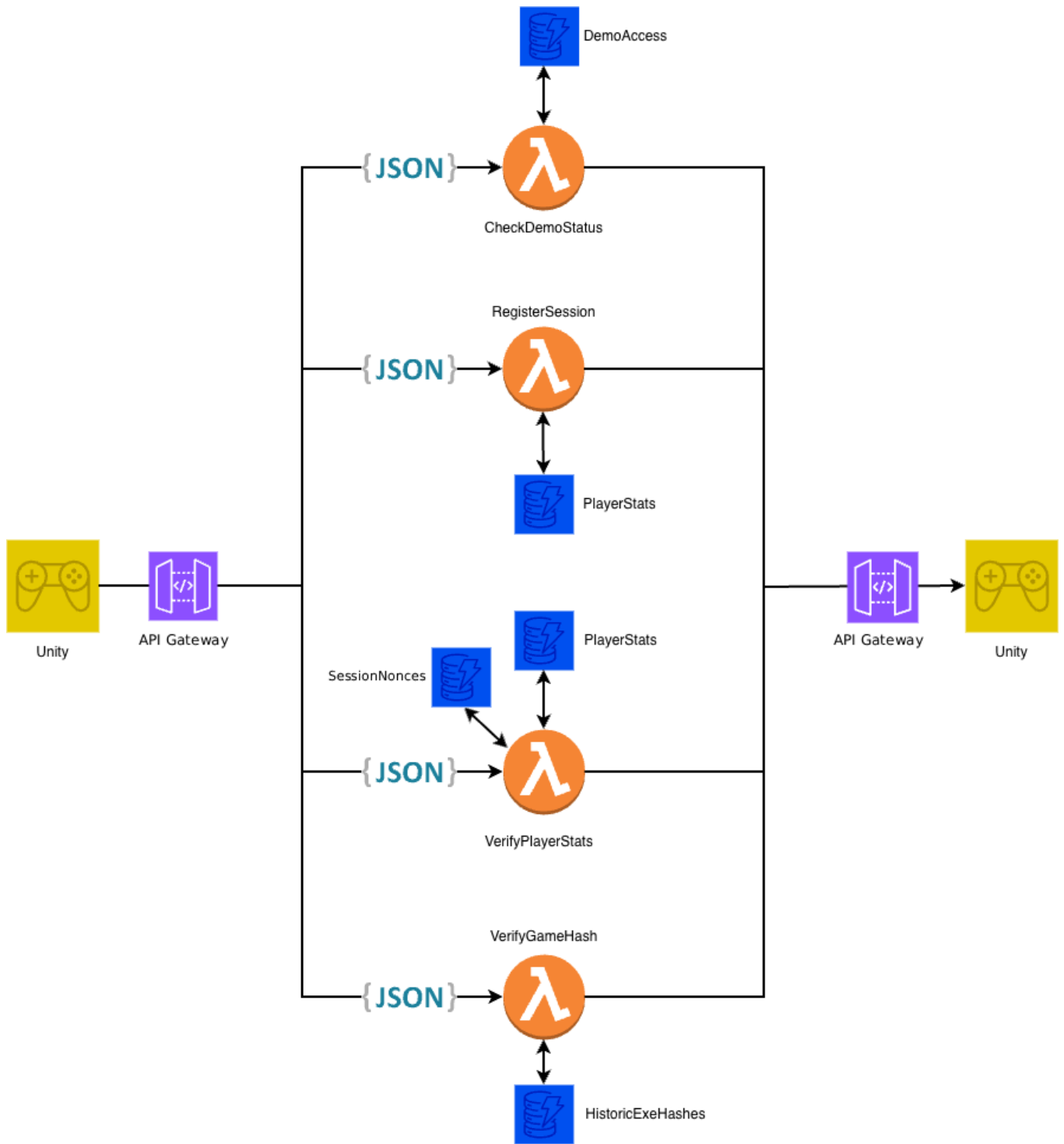


Figure 4.2: General view of the proposed *backend* architecture on AWS.

When a user accesses the web portal and signs in, *Cognito* verifies their credentials and provides the *JWT tokens* that identify them. With that *token*, the portal requests the download link from *API Gateway*; *Gateway* checks that the *token* is valid and passes the request to *Lambda*, which generates a pre-signed *S3* link with a short expiration window. The user receives that link directly in the browser and downloads the executable from *S3*, without exposing the *bucket* at any point or distributing credentials among clients. The flow diagram is shown in Figure 4.3.

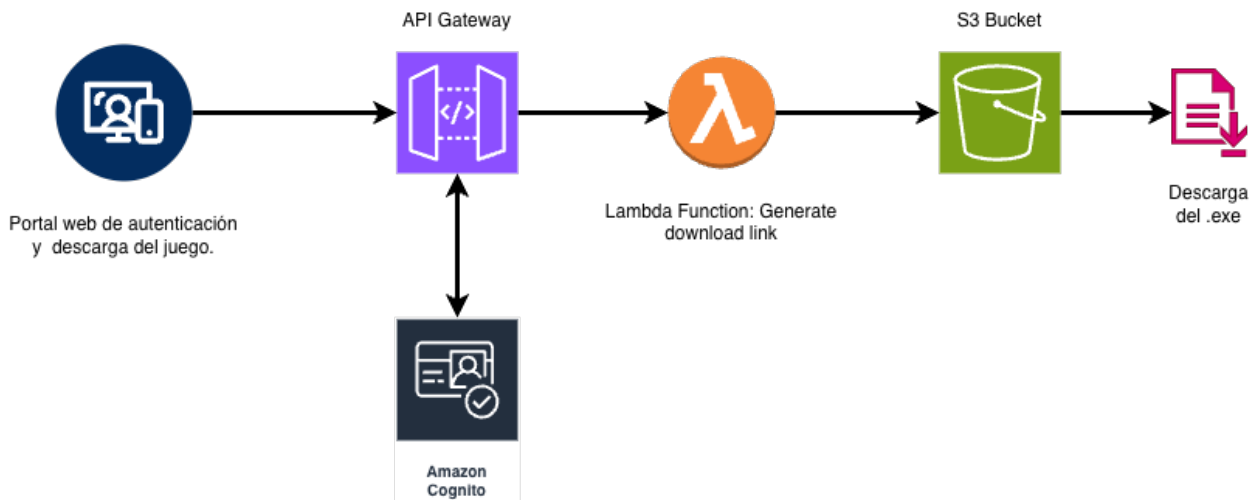


Figure 4.3: Authentication and download flow from the *web* portal.

4.2. Description of the main flows

4.2.1. VerifyPlayerStats flow: *Unity* → *AWS* → *Unity*

The verification process follows a client-server *request-response* pattern optimised for low latency. When a session starts, the client requests an ephemeral *nonce* from the server through an authenticated request with the user's *Cognito token*. The server generates a unique *UUID*¹ together with a 60-second *timestamp* and sends it to the client. The client verifies the *nonce*, the player telemetry (score, playtime), and the active session *token*, and sends them to *Lambda*.

Lambda validates the *nonce* atomically through a *ConditionExpression* that simultaneously checks that the *nonce* belongs to the authenticated user, has not been used previously, and has not expired. If one or more of these conditions fail, the request is rejected. This design transfers authentication entirely to the server: the client possesses no shared secret, so intercepting the *nonce* gives an attacker no guarantee and no ability to manipulate data.

If the client reports invalid or manipulated metrics, *Lambda* detects the anomaly, resets the statistics to zero, and activates the “cheater” flag in *DynamoDB*, returning the *CHEAT_WARNING* code to *Unity*. If the player repeats the offence, the user revocation protocol is executed: the account is disabled with *admin_disable_user* and the global

¹Universally Unique Identifier, a unique value used to identify an element unequivocally within a system.

session is closed with `admin_user_global_sign_out`, denying access to the account with the same username/email and also preventing future accounts from being created with the same credentials.

If the *nonce* is valid, the metrics sent by the user are validated against the data already stored in *DynamoDB*. Outdated metrics are rejected, prioritising the values already stored on the server. If the metrics match the server's metrics or represent only an insignificant difference, they are accepted as valid. *Lambda* confirms this to *Unity*, allowing *gameplay* to continue without additional intervention.

If the expected signature is not correct or local manipulation is detected, the function resets the statistics to zero, marks the cheater flag in *DynamoDB*, and sends *Unity* code 200 with the action BANNED. If the user cheats again and the cheater flag is already active in *DynamoDB*, the user *token* revocation protocol is activated, blocking future access or registration with that email or username.

For users without a previous record in *DynamoDB*, *Lambda* responds NEW_USER and the flow continues with data loading. Since there are no stored database records, they are created with initial values `Score=0` and `TimePlayed=0`.

Note: This client-assisted initialisation approach preserves *Zero Trust* because:

- *Lambda* maintains absolute read/validation authority.
- *Unity* only creates trivial records (zero values).
- All subsequent updates require validation in the *backend*.

Figure 4.4 shows the full statistics-validation process. It shows the decisions that *Lambda* makes based on the validity of the *nonce*, the session state, and the logical coherence of the data, as well as the response codes returned to *Unity* in each case. The FORCE_CLOUD state refers to a slight discrepancy between the client's local data and the data stored in *DynamoDB*; it is not necessarily manipulation, but only a minor desynchronisation. In this case, *Lambda* instructs *Unity* to discard the local state and adopt the server data without penalising the player.

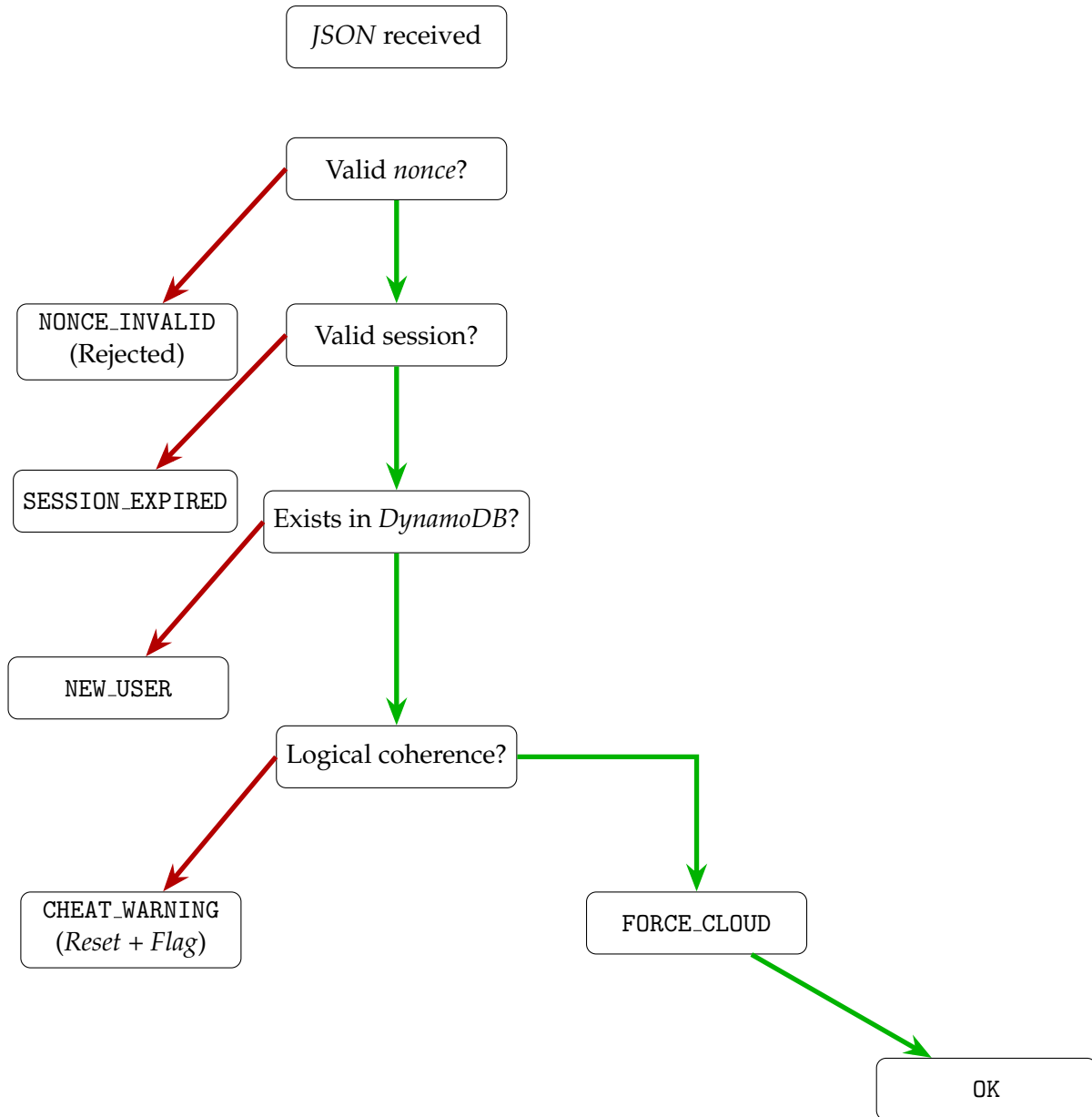


Figure 4.4: Player statistics validation process and inconsistency resolution.

4.2.2. VerifyGameHash flow: *Unity* → *AWS* → *Unity*

The check of whether the executable *hash* code is valid is performed in the *Lambda* function *VerifyGameHash*. The client sends a *JSON* containing the *hash* code; the function receives it and verifies whether it appears in the table where valid codes are stored.

When the code is received, it is compared with the codes in the *DynamoDB* table, leading to three possible scenarios:

- If the *hash* is not found among the table codes, which means that the executable has been modified, the error code *DENIED* is sent.
- If the code is found but does not have the flag indicating that it is the latest introduced version, an update-required code is sent. Since it appears in the table,

it is a *hash* that was introduced in a previous version and therefore does not imply executable manipulation.

- In the final case, the function has received a value that appears in the table and whose flag is active, returning OK and allowing the session on that executable.

Figure 4.5 illustrates how *Lambda* decides the response after receiving the executable *hash*. The function compares the incoming value with the `HistoricExeHashes` table and faces three scenarios: the *hash* does not appear in the table, in which case `DENIED` is returned because the executable is considered modified; the *hash* exists but does not have the current-version flag active, which is resolved with `UPDATE_REQUIRED` to force an update; or the *hash* matches the active version, at which point `OK` is returned and the session may continue. This logic makes it possible to maintain a history of legitimate versions and distinguish outdated clients from manipulated clients.

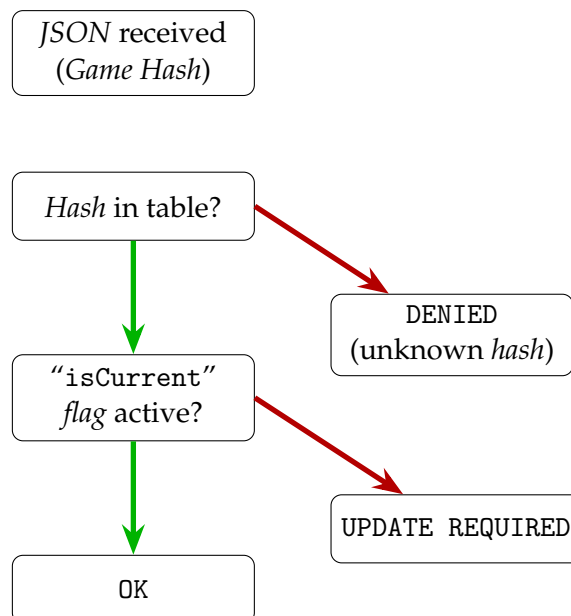


Figure 4.5: Executable-integrity verification process and valid-version management.

4.2.3. RegisterSession flow: *Unity* → *AWS* → *Unity*

`RegisterSession` applies a session-exclusivity system (anti multi-device), similar to those used by platforms such as *Netflix*, *HBO*, *Apple TV*, or *Spotify*: each user can only have one active session at a time. When the player enters or exits the game, *Unity* sends *AWS* a *JSON* with a session *token* (a unique identifier for that match) and an action indicating whether it wants to register (“register”) or release (“unregister”). From there, *Lambda* obtains the user identifier from the *Cognito token* and uses *DynamoDB* as a session-exclusion mechanism: the field that controls the session stores the active session *token*, or remains empty when the player has stopped playing.

Logout case: Releases the account by emptying the session-control field, allowing future logins. *Unity* executes this when the application closes.

Login case:

1. Verifies that the session *token* is included; if it is missing, responds 400 with ERROR: “Missing *token*”.
2. Queries the user’s data in *DynamoDB*.
3. If an active session already exists: blocks access with code 200, denies access, and records an intruder *log*.
4. If the account is free: assigns exclusivity by saving the new session *token*, records an access-granted *log*, and responds 200 with “OK Session registered”.

Figure 4.6 shows how *Lambda* processes a `RegisterSession` request according to the `action` field and the session state in *DynamoDB*. If the client sends `unregister`, the flow releases the session field and ends with OK. If the action is `register`, the function first checks that `sessionToken` is present (returning ERROR 400 if absent) and then queries the state in *DynamoDB*: if an active session already exists for that account, the request is rejected with `SESSION IN USE` and the attempt is logged; if the account is free, the new *token* is assigned through a conditional `UpdateItem` and the operation is confirmed with OK. This conditionality guarantees atomic session exclusivity even under simultaneous requests from multiple devices.

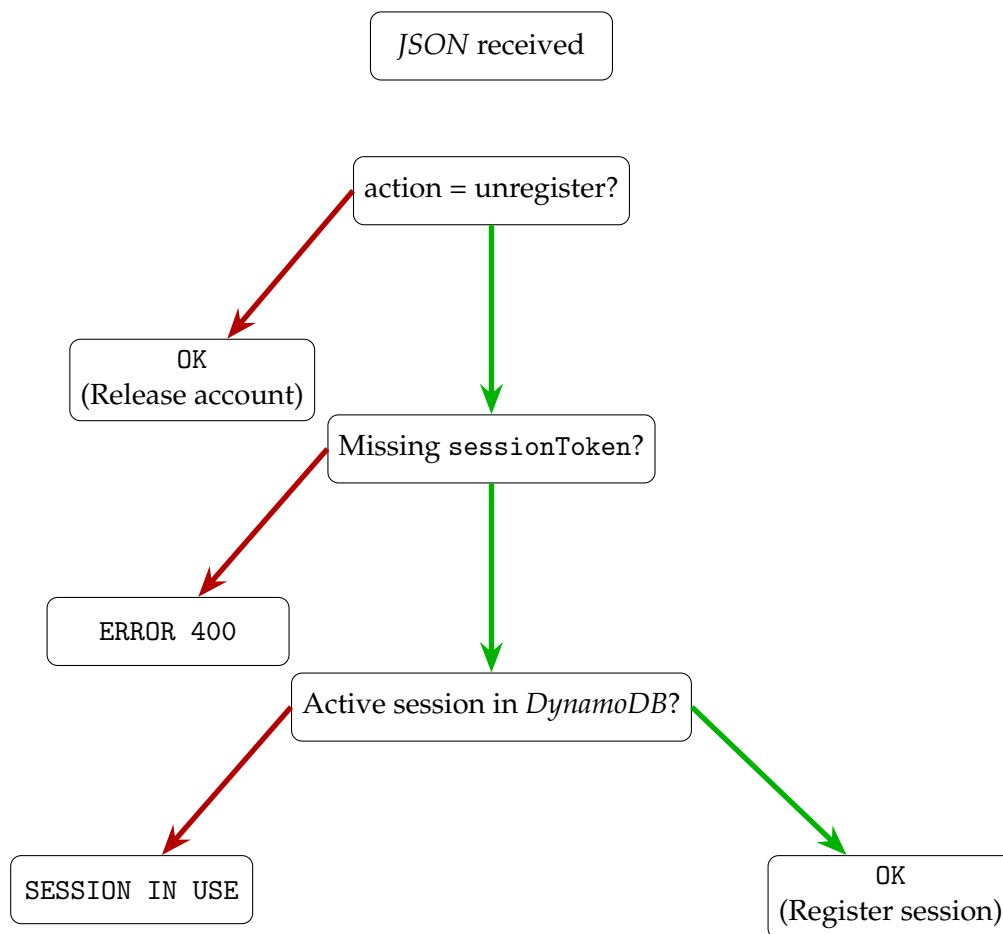


Figure 4.6: RegisterSession flow.

4.2.4. CheckDemoStatus flow: *Unity* → *AWS* → *Unity*

The CheckDemoStatus function is implemented to manage temporary access to the product, in this case a video game. After the first login or registration, the player has limited time during which the product is available.

After a player accesses the game for the first time with a new account, the function creates, for that user with a unique *ID*, a set of parameters in the *DynamoDB* table associated with the *Lambda* function. These parameters can be adjusted from the function or from the table itself. The data includes the first and last login instants and the last permitted instant, using Unix timestamps (*epoch*), that is, seconds elapsed since 1 January 1970 in *UTC*². There are also variables for total permitted playtime and used time, making it possible to manage available time by date, by playtime, or both.

Once the user is registered in the table, they have playtime, the possibility of playing until a specific date, or both. The function returns OK if the conditions for the session to remain available still hold, EXPIRED if entry is attempted after the expiration date, or TIME_LIMIT_REACHED if the available *demo* hours have been consumed.

Figure 4.7 shows the decision tree followed by *Lambda* when it receives a CheckDemoStatus request. The flow begins by checking whether the user already has a record in *DynamoDB*; if not, one is created with the initial parameters (first and last access instants, deadline, and permitted playtime) before evaluation continues. From there, the function checks two expiration conditions: if the deadline has passed, it returns EXPIRED; if the total playtime has been exhausted, it returns TIME_LIMIT_REACHED; otherwise, it returns OK, authorising the session to continue. This separation between date and accumulated time makes it possible to manage temporary access by calendar windows, by played hours, or by a combination of both.

²Coordinated Universal Time, the global time scale used as a reference for defining the rest of the time zones.

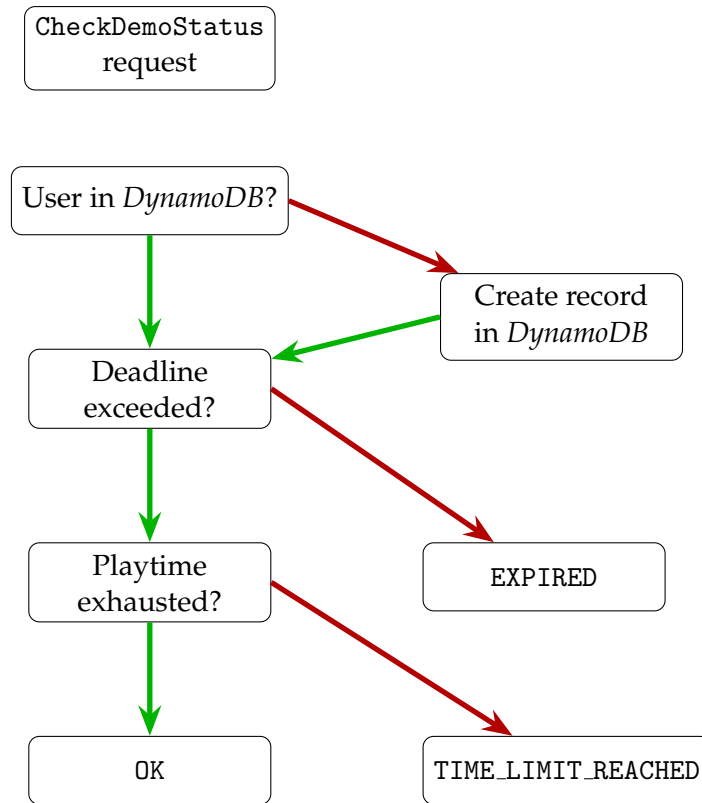


Figure 4.7: Temporary access-control flow in `CheckDemoStatus`.

4.2.5. User registration

The functions and the web page use the same *Cognito user pool*. All *Lambda* functions connect to *Unity* through *API Gateway*. These *APIs* use the aforementioned pool as authorizer.

The *web* page includes the option to register in order to create a new account. This section asks for a username, email address, and password. Once valid credentials are obtained, neither the user nor the email can already exist in the user database, and the password must meet minimum requirements and character rules; a verification code is then sent by email.

Once the account is activated through email verification, this becomes the account that identifies the user and with which they will sign in to the game. In this way, all functions have access to the users currently playing, allowing measures to be taken individually for each account.

4.2.6. GenerateDownloadLink flow: *Frontend* → *AWS* → *S3* → *Frontend*

`GenerateDownloadLink` creates secure temporary links to download the game or any other file. Its use is left to the developer. After signing in on the *launcher* website, the *frontend* requests the link by sending the session *token* to *AWS*. *Lambda* accesses *Cognito* to verify authentication and grant access to the *S3 bucket*.

The *Lambda* function already has the designated *bucket* from which the file must be served to the user. It accesses the *bucket* and creates a temporary link valid for 5 minutes (serialised field). It returns the direct link to the browser so the download can start immediately. If *Lambda* does not locate any file, it means that no file has been uploaded to the designated *bucket* or that the file type is not the expected one.

Security: Links are generated only for authenticated users; unauthorised access is blocked.

Cases:

1. Storage with package: locates the file, creates a temporary link, and confirms the download.
2. Storage without package: reports that no files are ready.
3. No authentication: blocks the request before processing.

Figure 4.8 represents the `GenerateDownloadLink` flow from the web portal to the S3 *bucket*. The request enters with the user's session *token*, and the function begins by verifying that the user is authenticated; if not, access is denied before touching storage. Once the identity is confirmed, *Lambda* accesses the designated *bucket* and checks whether the requested file exists: if none is found, it responds that no package is available; if it locates it, it generates a pre-signed *URL* with a short validity window and returns it directly to the browser to start the download. This pattern avoids distributing *bucket* credentials among clients and limits exposure of the file to the validity period of the link.

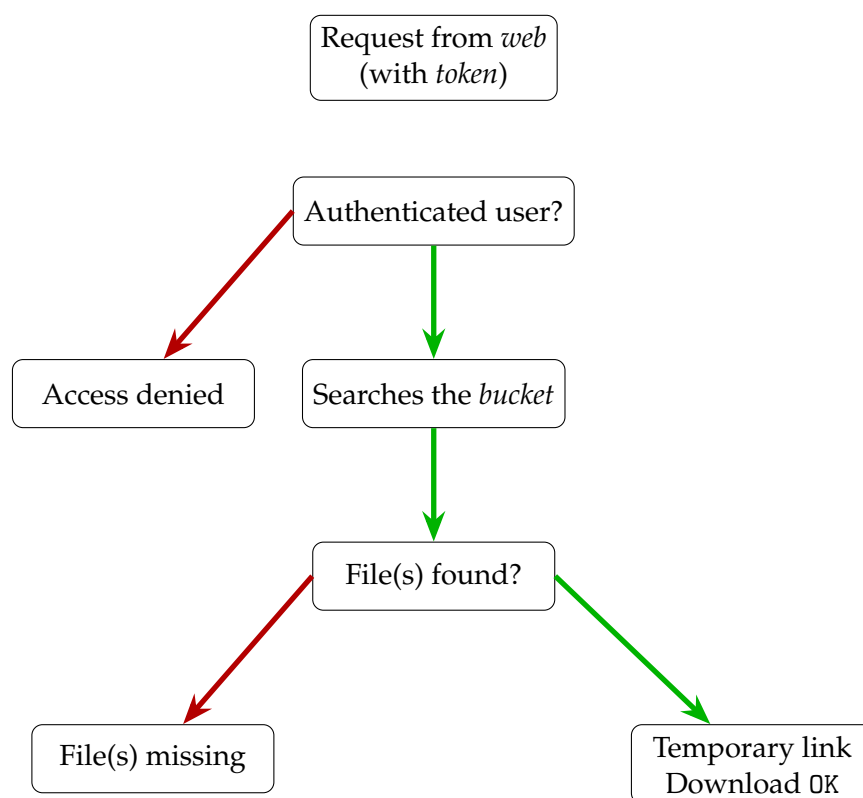


Figure 4.8: Temporary download-link generation flow for authenticated users.

4.3. Chapter conclusions

The proposed architecture centralises validation in a *serverless backend*, decoupling all critical logic from the *Unity* client. The implemented flows (*HMAC* verification, exclusive sessions, executable *hashes*, and temporary downloads) transfer authority to the server through *Zero Trust* principles. This architecture counters the possible vulnerabilities analysed in Chapter 2. It should be noted that *Unity*, as an engine oriented towards developer accessibility, presents a broader attack surface than other more restrictive environments. The relative ease with which reverse engineering can be applied means that placing critical logic on the client entails a risk [17], reinforcing the idea of centralising validation in the *backend*. This architecture also enables full traceability through *CloudWatch* for future audits if necessary. It supports the practical analysis in Chapter 5 by providing a replicable model for secure projects of many kinds.

Chapter 5. Use Cases and Observation

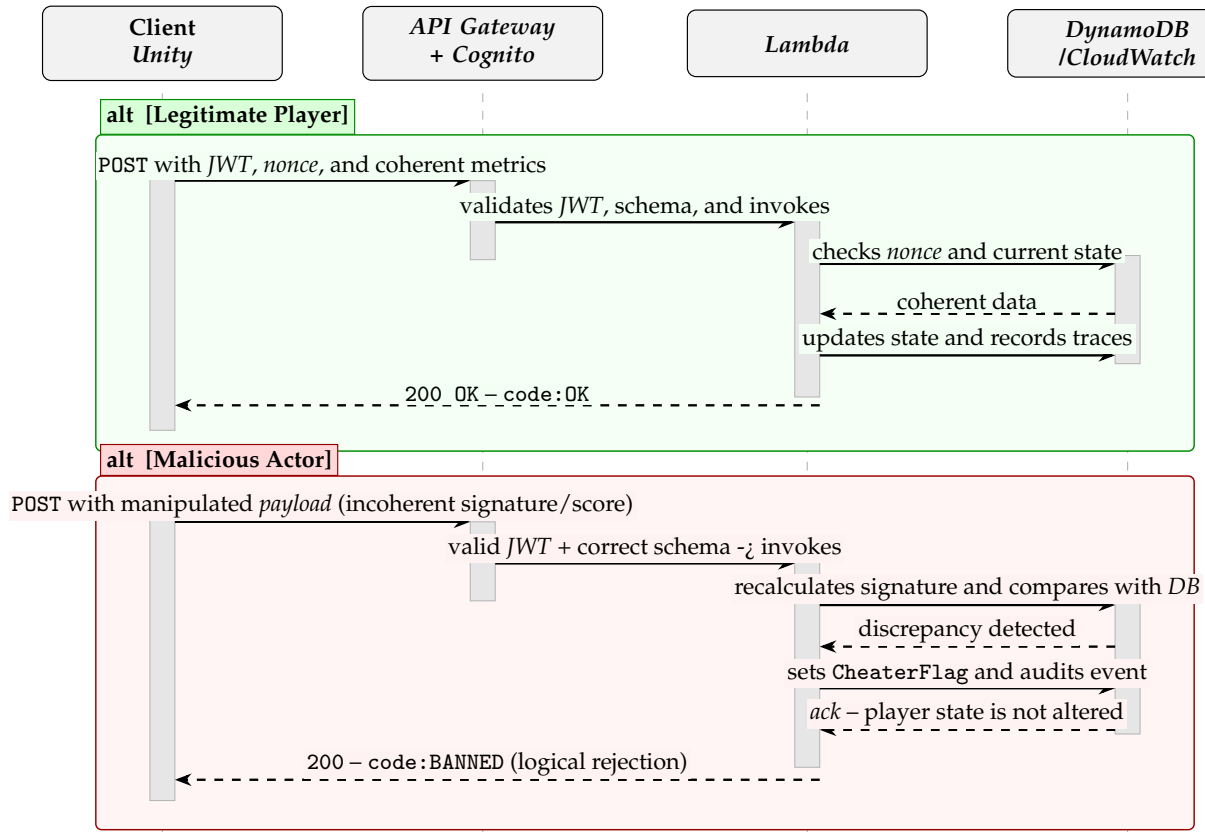


Figure 5.1: UML sequence diagram: legitimate player *vs.* malicious actor. Validation and transactional rejection flows in the *backend*.

5.1. Introduction and chapter objective

This chapter presents observations on the project architecture through a set of representative scenarios. Figure 5.1 introduces the contrast between a legitimate flow and a malicious attempt. The objective is to show the behaviour of the system when facing normal and adverse usage situations. In this way, it considers the centralisation of critical logic in the *serverless backend*.

The review is divided into four situations that are significant for the project: the behaviour of a legitimate player, data-manipulation attempts (*tampering*), management of the temporary or demo mode, and cases of desynchronisation between the client and the server. With these proposed scenarios, it is possible to verify whether the design decisions previously made fulfil their purpose in practice, taking into account data integrity, action traceability, and the responses given by the system when anomalous events occur.

In all described cases, observation relies on *API Gateway*, *Cognito*, *Lambda*, and *DynamoDB* routes, as well as on the logs generated by *CloudWatch*. In this way, the chapter shows how the system works, and together with Appendix 47 documents the evidence

through which the behaviour can be observed and verified. It also shows that the client plays a secondary role, since authority over sensitive data such as licences, sessions, or scores resides on the server.

5.2. Testing and observation methodology

The methodology of this chapter is based on executing several representative scenarios on the *backend* deployed on *AWS*. All cases are executed on the same environment, ensuring that observations are reproducible and that any subsequent iteration on the architecture can be replicated under the same conditions.

For the use cases, real requests are generated, covering both legitimate interactions and data-manipulation attempts. The data generated by these requests is observed through metrics and centralised *logs* in *CloudWatch*, as well as through queries on *DynamoDB* tables to verify data coherence after each scenario. The appearance of events marked as *WARN* or flags marking users as suspicious are some of the mechanisms through which fraudulent behaviour can be detected.

5.3. Use case 1: legitimate player

This first case analyses the system behaviour for a legitimate player. It explains the full flow from registration to completion of a match without any manipulation.

The player signs in, or registers and verifies their email, on the website, downloads the game, and accesses it using credentials managed by *Cognito*, obtaining valid *JWT tokens* for subsequent calls to the *backend*. During the match, *Unity* requests an ephemeral *nonce* that it will later send to *Lambda* together with the session telemetry data and events. These requests pass through *API Gateway*, which validates the signature, validity, and issuer of the *token*, as well as the *JSON* format. Finally, once the *nonce* has been validated, the *Lambda* functions register the scores and update the profiles, storing telemetry in *DynamoDB*.

CloudWatch logs show normal traffic, with correctly formed requests and no errors or warnings, as shown in Figure 5.2. The *DynamoDB* tables show correct updates linked to users without any alert flags, as observed in Figure 5.3, and *Lambda* returns *HTTP* 200 codes for each function.

Listing 5.1: Codes and bodies returned by the *Lambda* functions in a legitimate access

```
VerifyPlayerStats
  "statusCode": 200,
  "body": "{\"code\": \"OK\", \"message\": \"Synchronized.\"}"

VerifyGameHash
  "statusCode": 200,
  "body": "{\"code\":\"OK\", \"message\":\"Current version.\"}"

RegisterSession
  "statusCode": 200,
  "body": "{\"code\": \"OK\", \"message\": \"Session registered.\"}"
```



```

2026-05-01T11:07:19.802Z      START RequestId: 567e366d-069a-48f0-9ae9-9ce5ccb56cc9 Version: $LATEST
2026-05-01T11:07:19.827Z      === ITEM LEIDO: CheaterFlag=False PermanentBan=None ===
2026-05-01T11:07:19.867Z      === SCORES: local=0 db=0 | TIMES: local=0.00 db=0.00 ===
2026-05-01T11:07:19.907Z      END RequestId: 567e366d-069a-48f0-9ae9-9ce5ccb56cc9
2026-05-01T11:07:19.907Z      REPORT RequestId: 567e366d-069a-48f0-9ae9-9ce5ccb56cc9 Duration: 105.12 ms

```

Figure 5.2: *CloudWatch*: Log events from *VerifyPlayerStats*.

Userid (String) ▲	ActiveSessionToken ▼	CheaterFlag ▼	IPAddr... ▼	LastUpdated ▼	Score ▼	TimePlayed
a	<empty>			2026-05-01T...	6	7.72

Figure 5.3: *DynamoDB*: state of the *PlayerStats* table after a legitimate match.

This scenario confirms that the architecture works as expected under normal conditions, and also shows that authority over the game state resides in *AWS*, not in the client. The server maintains control over sensitive data even when the user behaves correctly.

5.4. Use case 2: *tampering*

This second use case evaluates how the system reacts when the client attempts to manipulate critical data, either by altering the executable, modifying the score *JSON*, or sending metrics that are inconsistent with the real playtime.

In a data-manipulation case, the flow begins in the same way as the legitimate case: the player authenticates with *Cognito*, obtains a valid *JWT token*, and plays a match. The difference appears when results are submitted. When the manipulated client generates a request to the score-submission *endpoint*, the request may pass structural *JSON Schema* validation in *API Gateway*, but contain an incoherent signature or score values disproportionate to the played time and contradictory to the history recorded in *DynamoDB*. When *Lambda* receives the request body, it recalculates the signature and compares it with the one provided by the client; if they do not match, or if the metrics violate the logical progression rules, *tampering* is considered to have occurred.

The *Lambda* function does not update the player's normal state. Instead, it records the incident in the corresponding *DynamoDB* table by setting a suspicious-user flag, resets or corrects the affected statistics, and writes a detailed audit event (with *UserID*, *IP*, *DeviceID*, and anomaly type) to the *CloudWatch* logs. From the perspective of *Unity*, the function responds with *HTTP 200* but includes a *BANNED* code in the body, so the client can react (blocking access, showing a message, or forcing logout) without distinguishing different *HTTP* codes. In repeated-offence scenarios, when *Lambda* detects that the cheater flag was already active for that user, the associated *tokens* are revoked, preventing future access from that account.

Listing 5.2: *CloudWatch* logs for first and second data manipulation by user "abc"

```

START RequestId : abefa9bf ...
=== ITEM READ : CheaterFlag = None PermanentBan = None ===
=== SCORES: local=100 db=0 | TIMES: local=0.00 db=0.00 ===
=== SUSPICIOUS DIFFERENCE: diff_score=100 diff_time=0.00 ===
=== CHEAT FLAG : abc | Reason : Manipulated score ===
=== DB VERIFICATION : CheaterFlag = True ===
END RequestId : abefa9bf ... Duration : 104.92 ms

--REPEATED OFFENCE--

START RequestId: 47b43a4b ...
=== ITEM READ: CheaterFlag=True PermanentBan=None ===
=== SCORES: local=100 db=0 | TIMES: local=0.00 db=0.00 ===
=== SUSPICIOUS DIFFERENCE: diff_score=100 diff_time=0.00 ===
Cognito disable OK: abc
Global sign-out OK: abc
=== PERMANENT BAN: abc | Reason: Manipulated score ===
END RequestId: 47b43a4b ... Duration: 572.79 ms

```

Figures 5.4 and 5.5 show the effect of this manipulation in *DynamoDB* and on the account state inside *Cognito*.

UserId (String) ▾	ActiveSessionToken ▾	CheaterFlag ▾	IPAddr... ▾	LastUpdated ▾	Score ▾	TimePlayed
abc	<empty>	true		2026-04-30T...	0	0

Figure 5.4: *DynamoDB*: state of the PlayerStats table after data manipulation.

User name	Email address	Email verified	Confirmation status	Status
abc		Yes	Confirmed	⊖ Disabled

Figure 5.5: *Cognito*: state of the *user pool* after repeated data manipulation.

If the executable signature does not match the one stored by the *backend*, the player is prevented from signing in. This discrepancy can occur in two cases: first, when the executable is not the latest version, and second, when the application has been modified.

CloudWatch logs would show the UPDATE_REQUIRED code in the case of an outdated *hash*, or DENIED depending on the case. At the *DynamoDB hash* storage level, these codes are saved manually and marked or unmarked as valid, as shown in Figure 5.6. With this distinction, *Lambda* sends one business code or another based on this variable.

clientHash (String)	isCurrent
e5693ea177cf9939a24f...	true
d37c3729afc1befbfe40...	false

Figure 5.6: *DynamoDB*: example of the *HistoricExeHashes* table with one valid code and one disabled code.

Listing 5.3: Examples of *JSONs* received by *Lambda*, and the codes and bodies returned by *VerifyGameHash*

```
JSON with enabled hash:
{"clientHash" : "e5693ea177cf9939a24f..."}

Lambda response:
{"statusCode": 200,
 "body": "{\"code\": \"OK\", \"message\": \"Session registered.\"}"

JSON with disabled hash:
{"clientHash" : "d37c3729afc1befbfe40..."}

Lambda response:
{"statusCode": 426,
 "body": "{\"code\": \"UPDATE_REQUIRED\", \"message\": \"Your version is not
the latest; update the game.\"}"

JSON with hash not belonging to the HistoricExeHashes table:
{"clientHash" : "invalidHashExample"}

Lambda response:
{"statusCode": 403,
 "body": "{\"code\": \"DENIED\", \"message\": \"This version is not
authorised.\"}"
```

This context validates that the verification layer in *Lambda* is effective. It is capable of detecting fraud and preventing corrupt data from being persisted by not updating the player's normal state, instead marking the user as suspicious. The technical response through business codes allows the client to react appropriately without the *backend* depending on the integrity of the player's device, thereby complying with the proposed *Zero Trust* model. Likewise, the ability to distinguish through the *hash* whether a discrepancy is due to an outdated version or to malicious executable modification allows specific responses to be applied in each case. This reinforces client-integrity control, ensuring that only authorised versions have access without transferring decision authority to the insecure user environment.

5.5. Use case 3: *demo* or limited mode

This use case evaluates the behaviour of the system when a player accesses the game with a *demo* session, that is, with the same authentication mechanisms as a normal user but subject to usage-time limits defined in the *backend*.

The flow begins when a new player signs in for the first time. After authentication, a record is created in the *DemoAccess* table in *DynamoDB*, which is associated with the *Lambda* function *CheckDemoStatus*. This table has configurable parameters such as the first and last login instants, the last permitted instant, the total available playtime, and the time already consumed. From that moment on, when *Lambda* queries and updates the table, it evaluates whether the user remains within the authorised access window by checking both whether the deadline has been exceeded and whether playtime is still available.

During execution, the codes returned by the function reflect state transitions, showing successful entries with OK or rejections through EXPIRED or TIME_LIMIT_REACHED. In addition, *DynamoDB* makes it possible to observe updates to player data whenever access succeeds, including played time, last available date, and other metrics. Figure 5.7 shows an example of that state.

userid (String) ▲	expirationEpoch ▼	firstLoginEpoch ▼	lastCheckEpoch ▼	playTimeSeconds ▼	usedSeconds
testUser1	1777804357	1777717718	1777718218	7200	293
testUser2	1777718345	1777716284	1777718325	7200	1510
testUser3	1777804367	1777709832	1777718362	7200	7503

Figure 5.7: *DynamoDB*: *DemoAccess* table at *epoch* date 1777718362.

Listing 5.4: Codes and bodies returned by *CheckDemoStatus* for users “testUser1”, “testUser2”, and “testUser3”, respectively

```
"statusCode": 200,
"body": "{\"code\":\"OK\",\"message\":\"Access allowed.\",\"allowed\":true,\"usedSeconds\":293,\"limitSeconds\":7200}"

"statusCode": 403,
"body": "{\"code\":\"EXPIRED\",\"message\":\"Demo expired.\"}"

"statusCode": 403,
"body": "{\"code\":\"TIME_LIMIT_REACHED\",\"message\":\"Demo time exhausted.\"}"
```

This use case confirms the robustness of the system in applying temporary restrictions without requiring client changes, since the *backend* centralises access logic. By delegating this check exclusively to *Lambda* and *DynamoDB*, the limitations are guaranteed to be unalterable by the end user, keeping demo control under a centralised access policy that responds correctly by interpreting the status codes returned by the server.

5.6. Use case 4: client-server desynchronisation

This use case analyses desynchronisation between the client’s local state and the state maintained by the server.

The scenario considers common situations such as an expired session, an obsolete game version, or match data that does not match the *backend* records. When *Unity* attempts to continue a match or send an action, the corresponding *Lambda* function

receives the request and executes the defined validations, detecting that the data is not valid according to the current system policies.

CloudWatch logs make it possible to reconstruct these discrepancies, identifying which user or session originated them and which correction policy the server applies in each event. The *logs* may show outdated requests with `FORCE_CLOUD` responses or 4XX errors, which facilitates debugging and auditing of these events through operational observability. Figure 5.8 shows the *DynamoDB* state used as a reference to force synchronisation with the cloud.

UserId (String) ▾	ActiveSessionToken ▾	Score ▾	TimePlayed
<u>S</u>	abc123-session-token	200	100

Figure 5.8: *DynamoDB*: example of user “S” in the *PlayerStats* table.

Listing 5.5: Outdated *JSON* received by *Lambda*, and the code and body returned by *VerifyPlayerStats*

```
User and body of the received JSON:
  "cognito:username": "S"
  "body": "{\"data\": \"{\\\"score\\\":0,\\\"timePlayedSeconds\\\":0}
  \\\",\\\"sessionToken\\\":\\\"abc123-session-token\\\",\\\"nonce\\\":
  \\\"550e8400-e29b-41d4-a716-446655440000\\\"}\"",

Response returned by Lambda:
  "statusCode": 200,
  "body": "{\"code\": \"FORCE_CLOUD\", \"message\": \"Outdated local
  game.\\\"}\""
```

This scenario represents typical situations such as an expired session, an expired *demo*, a client version that is no longer valid, or match data that has become obsolete compared with the server. In all cases, the principle is that the server acts as the source of truth and the client must adapt to the centralised state, preventing actions from being consolidated over inconsistent or outdated data.

5.7. Limitations and possible optimisations

Although the proposed architecture demonstrates the feasibility of transferring validation to the server and reinforcing the integrity of critical video game data, the project has several limitations derived from its scope and design decisions. The first is that the work focuses on logical security and state coherence, but does not address multiplayer networking problems. This fits the scope of the project, which excludes low-latency synchronisation and focuses on licences, progression, scores, and authentication.

Another important limitation is that, although binary *hash* control detects executable modifications, the client remains vulnerable to advanced techniques such as memory injection, network *proxies*, or reverse engineering that preserve the *hash* while altering logic at *runtime*. The architecture mitigates these risks through logical validations and telemetry, but it does not completely eliminate the client attack surface.

The dependence on managed AWS services such as *API Gateway*, *Lambda*, *DynamoDB*, *Cognito*, and *CloudWatch* should also be noted. This choice simplifies development, reduces operational burden, and facilitates scalability, but introduces a clear technological dependence on the provider and conditions the portability of the solution.

From an experimental standpoint, validation is performed on a prototype and on representative scenarios, not on a massive real-user load or a production environment. This means that the obtained results demonstrate the technical feasibility of the approach, but do not allow its behaviour under intense traffic peaks, sustained abuse, or extreme usage conditions to be extrapolated precisely. Consequently, the results obtained are useful as functional evidence, but they do not replace a large-scale performance-testing campaign.

Finally, the system protects specific assets such as licences, sessions, scores, *hashes*, and telemetry, but does not cover other risk vectors common in the industry, such as *kernel*-level protection, advanced detection of persistent *cheats*, or complete piracy prevention. These issues remain outside the scope by methodological decision and because they would require more invasive, less lightweight solutions that are less compatible with the project objective: offering a reproducible architecture applicable to independent or academic contexts.

5.8. Synthesis of results and validation limits

The tests performed made it possible to verify that the proposed *serverless* architecture works appropriately in the intended context. It was verified that validation can be transferred from the client to a controlled environment in *AWS*, centralising logic in *Lambda* functions and recording telemetry in *DynamoDB*. This helps protect critical data and detect anomalous behaviour. In addition, *CloudWatch* proves useful for system observability and auditing.

However, validation has some limitations. The tests were performed in controlled scenarios, so they do not ensure performance under massive loads or *DDoS* attacks. In addition, the client remains vulnerable to techniques such as reverse engineering or memory injection. The proposal focuses on logical security and state coherence, without addressing problems specific to massive multiplayer games, such as latency or real-time synchronisation.

Figure 5.9 shows latency by *Lambda* function and the distribution of events detected in *CloudWatch* during the previous scenarios. The figures come only from those executions.

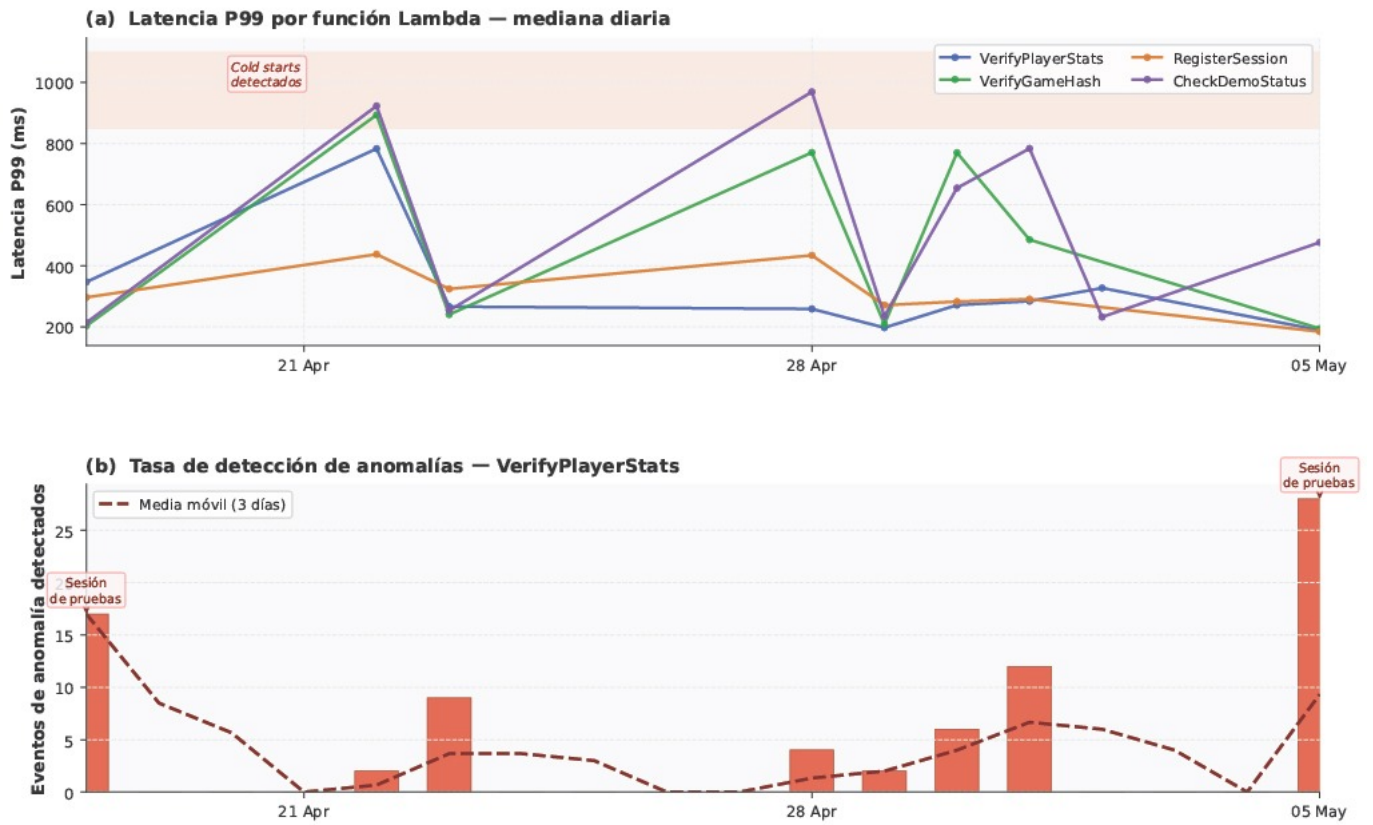


Figure 5.9: P99 latency by *Lambda* function (a) and anomaly-detection rate in *VerifyPlayerStats* (b). Data exported from *Amazon CloudWatch*. Period: 18 Apr - 5 May 2026.

Chapter 6. Conclusions and Future Work

This work has fulfilled the objective of designing a *serverless* architecture that shifts validation authority from the client to the server. After implementing, deploying, designing, and observing the architecture across the use cases described in Chapter 5, the following conclusions have been drawn:

- **Validation of the paradigm shift:** After testing and examining the legitimate player and data-tampering scenarios (Use Cases 1 and 2), it can be noted that the *Unity* client can be limited to a display role, while key data and decisions concerning sessions, records, scores, and licences reside in and are resolved by *AWS*. The *CloudWatch* traces collected show that scores persisted in *DynamoDB* always come from validation in *Lambda*, and not from the value reported by *Unity* (the player).
- **Sustainability and scalability:** With the *serverless* model, an architecture has been achieved that allows the system to scale automatically according to player demand, thereby reducing costs. This makes the model suitable for independent studios and academic projects.
- **Observability as a core component of security:** The combination of *DynamoDB* and *CloudWatch* has made it possible to build an auditable record of system interactions, not only of critical interactions. The traces collected in Chapter 5 and in the appendices show that telemetry not only facilitates debugging for developers, but also provides the developer and the platform with verifiable traces in the face of tampering attempts. This enables automated responses, previously programmed in *Lambda*, such as activating the *CheaterFlag*, invalidating statistics manipulated by the player, or revoking *tokens* via `admin.disable_user` and `admin_user_global.sign_out` in cases of repeated offences.
- **Effectiveness of the Zero Trust model:** The combination of *API Gateway* and *Cognito* rejected, in the scenarios considered, unauthenticated requests, requests with an invalid *token*, and requests that did not comply with the expected *JSON*, before they reached the business logic. This behaviour is consistent with the hypothesis formulated in the introduction and with the acceptance criteria defined in Chapter 3.

Future work

The proposed architecture covers the objectives set out in this Final Degree Project, although it leaves open several lines of research and extension that would consolidate and broaden the results obtained. These lines have been grouped around three axes: improving detection capabilities, the quantitative characterisation of the system, and the consolidation of the prototype as a replicable model.

- **Proactive detection through *Machine Learning*:** Transitioning from reliable but static observability to an intelligent system that uses machine-learning models to identify complex fraud patterns and generate better data for subsequent auditing. By taking advantage of the large volume of data collected through telemetry, *AI* could detect anomalies that the current static logic cannot.
- **Operational cost analysis:** A future line of work would be to study the real cost of the system when used over a prolonged period, identifying which services weigh most heavily on the bill (*Lambda*, *DynamoDB*, or *CloudWatch*) and proposing adjustments to reduce it. This would reinforce the idea that the architecture is affordable for small studios and academic projects.
- **Extension to real-time multiplayer networks:** Another future line would be to address security in multiplayer networks. This would involve investigating how server authority could be maintained in flows requiring low latency, evaluating alternatives such as *AWS GameLift*, so as not to compromise the player's experience. The viability of this service in production is supported by its use in titles such as *Apex Legends*, *Dead by Daylight*, and *Mortal Kombat 1*, as well as by its demonstrated capacity to scale up to 100 million concurrent players [18].
- **Creation of a replicable *framework*:** The objective would be to move from the current prototype to a fully replicable model that allows small teams to adopt "security by design" principles efficiently and at a reduced operational cost.

6.1. Individual Contributions

The distribution of responsibilities during the implementation phase was organised around the *Lambda* functions deployed in *AWS*.

6.1.1. Dorje Khampa Herrezuelo Blasco

- **GenerateDownloadLink:** generates temporary pre-signed S3 links from the user's *Cognito* token, allowing the executable to be downloaded from the *launcher* website without exposing the *bucket* credentials.
- **RegisterSession:** ensures session exclusivity per user through *DynamoDB*, managing the *login* and *logout* flows in order to prevent simultaneous sessions on different devices.
- **VerifyPlayerStats:** core of the anti-fraud system. It validates the *nonce* of each match, compares the client metrics with *DynamoDB*, and activates response protocols when tampering is detected, such as the *CheaterFlag*, the reset of manipulated statistics and, in cases of repeated offences, credential revocation.

6.1.2. Alberto Peñalba Martos

- Development of everything related to *Cognito* and its integration with *Unity*, including user authentication, *JWT token* management, and the connection between the game client and the identity system shared with the web portal.
- `VerifyGameHash`: verifies the integrity of the executable by comparing its *hash* against the `HistoricExeHashes` table, distinguishing between tampered versions (`DENIED`), outdated versions (`UPDATE.REQUIRED`), and current valid versions (`OK`).
- `CheckDemoStatus`: controls temporary access to the product through a date window and accumulated playtime, returning `OK`, `EXPIRED`, or `TIME_LIMIT_REACHED` depending on the user's status.

6.1.3. Joint Work

The overall system architecture, the data model in *DynamoDB*, and the shared *HTTP* response convention used by all functions were defined jointly. The team also developed the *launcher* web portal and a 2D game prototype in *Unity*, which was used to validate the integration with *AWS* and verify the behaviour of the functions in the use cases described in Chapter 5. The writing of the report and the creation of the figures were also carried out collaboratively.

6.2. Repository

The source code of all the *Lambda* functions can be found at the following link: <https://github.com/DorjeeHerBlas/TFG-25-26-AWS>

Appendices

Appendix A: *Lambda* Function `VerifyPlayerStats`

Phase 1 — Clean startup

First startup after clearing the record. The *nonce* is issued and consumed correctly, and verification confirms that the player has no active *flags* and that the local *score* is coherent with the cloud value.

Listing 6.1: *CloudWatch* and *Lambda* — clean startup

```
START RequestId: efdcdaa4 ... Init Duration: 628.79 ms
START RequestId: 715f33fc ...
=== ITEM READ: CheaterFlag=None PermanentBan=None ===
=== SCORES: local=2 db=0 | TIMES: local=4.60 db=5.00 ===
END RequestId: 715f33fc ... Duration: 129.40 ms

Response returned by Lambda:
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\": \"OK\", \"message\": \"Synchronized.\"}"
}
```

Phase 2 — First offence (CHEAT_WARNING)

The user has manipulated the data, and the `VerifyPlayerStats` function detected it by comparing the metrics with those already stored in *DynamoDB*. `CheaterFlag` is activated, but access is not revoked because this is not a repeated offence.

Listing 6.2: *CloudWatch* and *Lambda* — first offence

```
START RequestId: 12f78bdf ...
=== ITEM READ: CheaterFlag=None PermanentBan=None ===
=== SCORES: local=18 db=7 | TIMES: local=22.43 db=19.55 ===
=== SUSPICIOUS DIFFERENCE: diff_score=11 diff_time=2.88 ===
=== CHEAT FLAG: d | Reason: Manipulated score ===
=== DB VERIFICATION: CheaterFlag=True ===
END RequestId: 12f78bdf ... Duration: 104.92 ms

Response returned by Lambda:
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\": \"CHEAT_WARNING\", \"message\": \"Warning: manipulated score. Stats reset.\"}"
}
```

Phase 3 — Startup with active *flag*

The player has signed in correctly, sending unmanipulated metrics, but `CheaterFlag` remains active. *Lambda* returns the same *output* as in Phase 1.

Listing 6.3: *CloudWatch* — startup with active *flag*

```
START RequestId: 2b77c88f ...
=== ITEM READ: CheaterFlag=True PermanentBan=None ===
=== SCORES: local=0 db=0 | TIMES: local=24.54 db=0.00 ===
END RequestId: 2b77c88f ... Duration: 100.77 ms
```

Phase 4 — Repeated offence: permanent *ban*

The user sends manipulated metrics again. Since `CheaterFlag` is active, access is revoked by setting `PermanentBan` to true, and *Lambda* proceeds to disable the user.

Listing 6.4: *CloudWatch* and *Lambda* — permanent *ban* for repeated offence

```
START RequestId: 8633fb92 ...
=== ITEM READ: CheaterFlag=True PermanentBan=None ===
=== SCORES: local=100 db=0 | TIMES: local=13.03 db=24.54 ===
=== SUSPICIOUS DIFFERENCE: diff_score=100 diff_time=-11.52 ===
Cognito disable OK: d
Global sign-out OK: d
=== PERMANENT BAN: d | Reason: Manipulated score ===
END RequestId: 8633fb92 ... Duration: 572.79 ms

Response returned by Lambda:
{
  "statusCode": 403,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\": \"PERMANENT_BANNED\", \"message\": \"Permanent ban: manipulated score\"}"
}
```

PlayerStats **table**

The following table is an example of the data that `PlayerStats` could contain. It is used both to query and update data by the *Lambda* functions `VerifyPlayerStats` and `RegisterSession`.

The columns contain: the *username*, the *token* indicating whether the account is currently active, a *flag* to mark accounts that have manipulated their data, the *IP* address, the last record, the score, and the playtime.

UserId	ActiveSessionToken	CheatFlag	IPAddress	LastUpdated	Score	Time
a	f221d0ea-ad3a...	false	XX.XX.XX	2026-04-29T...	3	3.12
b	<empty>	false	XX.XX.XX	2026-04-29T...	10	16.5
c	<empty>	true	XX.XX.XX	2026-04-29T...	9	14.6
d	<empty>	false	XX.XX.XX	2026-04-29T...	13	16.5
e	<empty>	false	XX.XX.XX	2026-04-29T...	71	103.9

Table 6.1: Example state of the *DynamoDB* table *PlayerStats*.

SessionNonces table

The following table shows the state of *SessionNonces* with three accounts signed in over a very short period. This table stores the ephemeral *nonces* that *VerifyPlayerStats* checks to ensure that each one belongs to the corresponding user and has not already been used.

The table shows the *nonce* code, expiration time or *TTL*, whether the *nonce* has already been used, and the *username*.

nonce	expiresAt	used	userId
1ba8e285-3e66...	1777540990	true	a
5df6a064-11a6...	1777540971	true	b
bddbb0ea-1004...	1777541023	true	c

Table 6.2: Example state of the *DynamoDB* table *SessionNonces*.

Appendix B: *Lambda* Function *RegisterSession*

Execution traces (*CloudWatch*)

The *logs* show two different scenarios: legitimate accesses in a normal session and blocks caused by an attempted concurrent access from another device.

Scenario 1 — Consecutive legitimate accesses

Multiple startups by the same user *d* in successive sessions. In each case, the previous *ActiveSessionToken* had been released when the game closed, so the *Lambda* function grants access and records the new registered *token* again in the *PlayerStats* table in *DynamoDB*.

Listing 6.5: *CloudWatch* and *Lambda* — consecutive legitimate accesses

```
START RequestId: 3ad09b93 ... Init Duration: 490.36 ms
Access granted to d. Gate locked.
END RequestId: 3ad09b93 ... Duration: 435.26 ms

START RequestId: 794f6630 ...
Access granted to d. Gate locked.
END RequestId: 794f6630 ... Duration: 67.47 ms

START RequestId: 6417f039 ...
Access granted to d. Gate locked.
END RequestId: 6417f039 ... Duration: 80.24 ms
```

```

Response returned by Lambda in all three cases:
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\": \"OK\", \"message\": \"Session registered.\"}"
}

```

Scenario 2 — Block due to session in use

User a starts the game, and while the session remains active, an attempt is made to sign in from a second device. When this occurs, *Lambda* detects that `ActiveSessionToken` is not empty and rejects access with `SESSION_IN_USE`. The first device continues playing without interruption. Only when the token is released (closing the game on the first device) is the second access granted.

Listing 6.6: *CloudWatch* and *Lambda* — block due to session in use

```

START RequestId: 786352f2 ...
Access granted to a. Gate locked.
END RequestId: 786352f2 ... Duration: 102.65 ms

START RequestId: 1922f9c6 ...
INTRUDER BLOCKED: a attempted to enter from another PC.
END RequestId: 1922f9c6 ... Duration: 72.43 ms

Response returned by Lambda:
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\": \"SESSION_IN_USE\", \"message\": \"There is already
a match in progress on another device.\"}"
}

START RequestId: ee75bb39 ...
Access granted to a. Gate locked.
END RequestId: ee75bb39 ... Duration: 74.98 ms

```

Scenario 3 — Multiple users in parallel

The latest records show accesses by user “abc” interleaved with those by user “d”, confirming that the system manages sessions independently per account without interference between different users.

Listing 6.7: *CloudWatch* — parallel sessions from different users

```

START RequestId: affde188 ...
Access granted to abc. Gate locked.
END RequestId: affde188 ... Duration: 46.23 ms

START RequestId: 98acc28a ...

```



```
Access granted to d. Gate locked.
END RequestId: 98acc28a ... Duration: 93.49 ms
```

Appendix C: *Lambda* Function `VerifyGameHash`

Execution traces (*CloudWatch*)

The *logs* show three different scenarios: access with a non-existing *hash*, access with an outdated executable, and access with the latest version of the executable.

Scenario 1 — Non-existing *hash*

All *requests* from the test session return `INFO undefined` in the *log*, which corresponds to `console.log(result.Item)` in the code when the *hash* sent by the client does not exist in the `HistoricExeHashes` table. `result.Item` is undefined, and the *Lambda* function responds with code `DENIED`. This occurs because, during testing, the executable is launched from the *Unity* editor, whose *hash* is not registered as a valid published version.

Listing 6.8: *CloudWatch* and *Lambda* — unregistered *hash*

```
START RequestId: c028133e ... Init Duration: 425.04 ms
INFO undefined
END RequestId: c028133e ... Duration: 888.03 ms

Response returned by Lambda:
{
  "statusCode": 403,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\":\"DENIED\",\"message\":\"This version is not authorised.\"}"
}
```

Scenario 2 — Outdated *hash*

The *log* shows both the *hash* and the fact that it is not current. This occurs because an old version of the program has been executed; therefore, the *hash*, which no longer corresponds to the latest version, has not been deleted but its *isCurrent* *flag* has been unchecked. In this scenario, *Lambda* responds with code `UPDATE_REQUIRED`.

Listing 6.9: *CloudWatch* and *Lambda* — outdated *hash*

```
START RequestId: 2b47d6b7 ...
isCurrent: { BOOL: false }, clientHash: { S: 'c2f3d552 ...' }
END RequestId: 2b47d6b7 ... Duration: 803.63 ms

Response returned by Lambda:
{
  "statusCode": 426,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\":\"UPDATE_REQUIRED\",\"message\":\"Your version is not"}
```

```
the latest; update the game.\"}"
}
```

Scenario 3 — Correct access

The listing for this last possible scenario shows the same information as the previous case, except that the *isCurrent flag* is valid, indicating a valid record from an updated and unmodified executable.

Listing 6.10: *CloudWatch* and *Lambda* — valid access

```
START RequestId: 7e59a00b ...
isCurrent: { BOOL: true }, clientHash: { S: '62fbbd30 ...' }
END RequestId: 7e59a00b ... Duration: 65.66 ms

Response returned by Lambda:
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\":\"OK\",\"message\":\"Current version.\"}"
}
```

HistoricExeHashes table

The following is an example of the contents of the *HistoricExeHashes* table. The *VerifyGameHash* function checks this data and performs the necessary checks to return the appropriate code.

In order, it shows the historical list of *hashes* and whether each one is currently valid.

clientHash	isCurrent
62fbbd309...	true
c2f3d552a...	false
0948dc407...	false

Table 6.3: Example state of the *DynamoDB* table *HistoricExeHashes*.

Appendix D: *Lambda* Function CheckDemoStatus

This section shows the possible access cases for the game with a *demo* version. The three possible responses are: a session that exceeds the deadline, access from an account with more playtime than allowed, or a game session that satisfies the conditions.

Scenario 1 — Expired demo validity date

The shown *request* indicates that the attempt to play occurred on a date later than the one defined for that user. The numbers displayed are *epochs*; the first corresponds to the current date and the second to the deadline. In this scenario, *Lambda* returns code EXPIRED.

Listing 6.11: *CloudWatch* and *Lambda* — deadline reached

```
START RequestId: 9b8518d8 ...
Date expired: 1777469484 > 1776555512
END RequestId: 9b8518d8 ... Duration: 48.77 ms

Response returned by Lambda:
{
  "statusCode": 403,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\":\"EXPIRED\",\"message\":\"Demo expired.\"}"
}
```

Scenario 2 — Allowed time exceeded

In this case, the account attempting access has consumed the two hours it was granted. The numbers shown are the consumed seconds and the seconds limit. In this scenario, *Lambda* returns code `TIME_LIMIT_REACHED`.

Listing 6.12: *CloudWatch* and *Lambda* — demo time exceeded

```
START RequestId: f4e317c2 ...
Time limit reached: 7205 > 7200
END RequestId: f4e317c2 ... Duration: 157.24 ms

Response returned by Lambda:
{
  "statusCode": 403,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\":\"TIME_LIMIT_REACHED\",\"message\":\"Demo time exhausted.\"}"
}
```

Scenario 3 — Valid session

When the player accesses the *demo* within the date and playtime parameters, the *CloudWatch logs* show the beginning and end of each request, without additional information. Since the session is validated, *Lambda* returns code 200.

Listing 6.13: *CloudWatch* and *Lambda* — valid session

```
START RequestId: 85c2cdfb ...
END RequestId: 85c2cdfb ... Duration: 191.86 ms

Response returned by Lambda:
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\":\"OK\",\"message\":\"Access allowed.\",\"allowed\":
```

```
    true, \"usedSeconds\":0, \"limitSeconds\":7200}"
  }
```

DemoAccess table

The following table shows a possible state of the data contained in DemoAccess, the table from which the *Lambda* function CheckDemoStatus queries and updates data.

In order, it shows the user *ID*, the *demo* expiration date, the first and last login times (in *epoch*), the time limit, and the time used.

userId	expiration	firstLogin	lastCheck	playTime	usedTime
302c29fc...	1777450207	1777453515	1777465999	7200	332
20cc19dc...	1777539496	1777453096	1777453127	7200	31
605c496c...	1777476445	1777390045	1777463377	7200	3949
602ca93c...	1777476933	1777390533	1777475205	7200	2291
a01c793c...	1777463067	1777406667	1777474763	7200	1010

Table 6.4: Example state of the *DynamoDB* table DemoAccess.

Web page link

The following [link](#) asks you to create an account with which you can download the game. Once the *Unity* executable opens, it can be tested with the same account.

Bibliography

- [1] Newzoo, “Global games market report 2022,” 2022, [Online]. Available: <https://newzoo.com/resources/trend-reports/newzoo-global-games-market-report-2022-free-version>.
- [2] J. Prazak, M. Macas, and T. Cerny, “New usage of telemetry for anti-cheating in fps game,” 2024, [Online]. Available: https://www.researchgate.net/publication/377645285_New_usage_of_telemetry_for_anti-cheating_in_FPS_game.
- [3] OWASP Foundation, “Owasp api security top 10,” 2023, [Online]. Available: <https://owasp.org/API-Security/>.
- [4] Amazon Web Services, *Amazon Cognito Developer Guide*, 2023, [Online]. Available: <https://docs.aws.amazon.com/cognito/>.
- [5] Denuvo Software Solutions, “Denuvo anti-tamper,” 2025, [Online]. Available: <https://irdeto.com/denuvo/>.
- [6] I. Bogost, “The drm debate in pc gaming: Denuvo’s performance and player perception,” *Game Studies Journal*, vol. 18, no. 2, 2018, [Online]. Available: <http://gamestudies.org/1802>.
- [7] BattlEye Innovations, “Battleye anti-cheat,” 2025, [Online]. Available: <https://www.battleye.com/>.
- [8] Amazon Web Services, *Security Overview of AWS Lambda*, 2023, [Online]. Available: <https://docs.aws.amazon.com/lambda/latest/dg/security.html>.
- [9] —, *Controlling and Managing Access to REST APIs in API Gateway*, 2023, [Online]. Available: <https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-control-access-to-api.html>.
- [10] —, *Security Best Practices for Amazon DynamoDB*, 2023, [Online]. Available: <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/security-best-practices.html>.
- [11] Center for Internet Security, “Cis amazon web services foundations benchmark, v1.4.0,” 2021, [Online]. Available: https://www.cisecurity.org/benchmark/amazon_web_services.
- [12] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” National Institute of Standards and Technology (NIST), Special Publication 800-207, 2020, [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/specialpublications/NIST.SP.800-207.pdf>.
- [13] OWASP Foundation, “Owasp top 10:2021 – the ten most critical web application security risks,” 2021, [Online]. Available: <https://owasp.org/Top10/>.

- [14] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," *ACM Transactions on Computer Systems*, vol. 2, no. 4, pp. 277–288, 1984. [Online]. Available: <https://web.mit.edu/saltzer/www/publications/endtoend/endtoend.pdf>
- [15] Amazon Web Services, "How API Gateway resource policies affect authorization workflow," 2024. [Online]. Available: <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-authorization-flow.html>
- [16] H. Siamak and T. Mortaza, "Owasp secure by design framework," <https://owasp.org/www-project-secure-by-design-framework/>, [Accessed 04-12-2025]. [Online]. Available: <https://owasp.org/www-project-secure-by-design-framework/>
- [17] F. Nightingale, "Unity real-time development platform vulnerability let attackers execute arbitrary code." [Online]. Available: <https://cybersecuritynews.com/unity-real-time-development-platform-vulnerability/>
- [18] M. Jackson, D. Holladay, and J. Jantunen, "Apex legends migrates to Amazon GameLift servers in just 10 days," May 2025, [Accessed 26-04-2026]. [Online]. Available: <https://aws.amazon.com/blogs/gametech/apex-legends-migrates-to-amazon-gamelift-servers-in-just-10-days/>